

SKALA

Vector DB

윤재성

SK 주식회사 AX



1. 비정형 데이터와 텍스트 수치화 기법
2. 벡터 유사도와 **Vector DB**의 등장
3. **RAG** 아키텍처와 **Qdrant** 핵심 기능
4. 실습
5. 종합실습



1. 비정형 데이터와 텍스트 수치화 기법

SK 주식회사 AX

1. 비정형 데이터와 텍스트 수치화 기법

- 정형 데이터와 비정형 데이터

SK 주식회사 AX

정형 데이터와 비정형 데이터

◉ 정형 데이터 (Structured Data)

- 엑셀 시트나 데이터베이스 테이블처럼 가로줄(행)과 세로줄(열)이 엄격하게 정해져 있는 형태입니다.
- 나이, 가격, 입사일, 회원 번호처럼 각 칸마다 어떤 종류의 값(숫자, 날짜, 참/거짓 등)이 들어갈지 미리 약속되어 있습니다.

◉ 정형 데이터 구조 예시







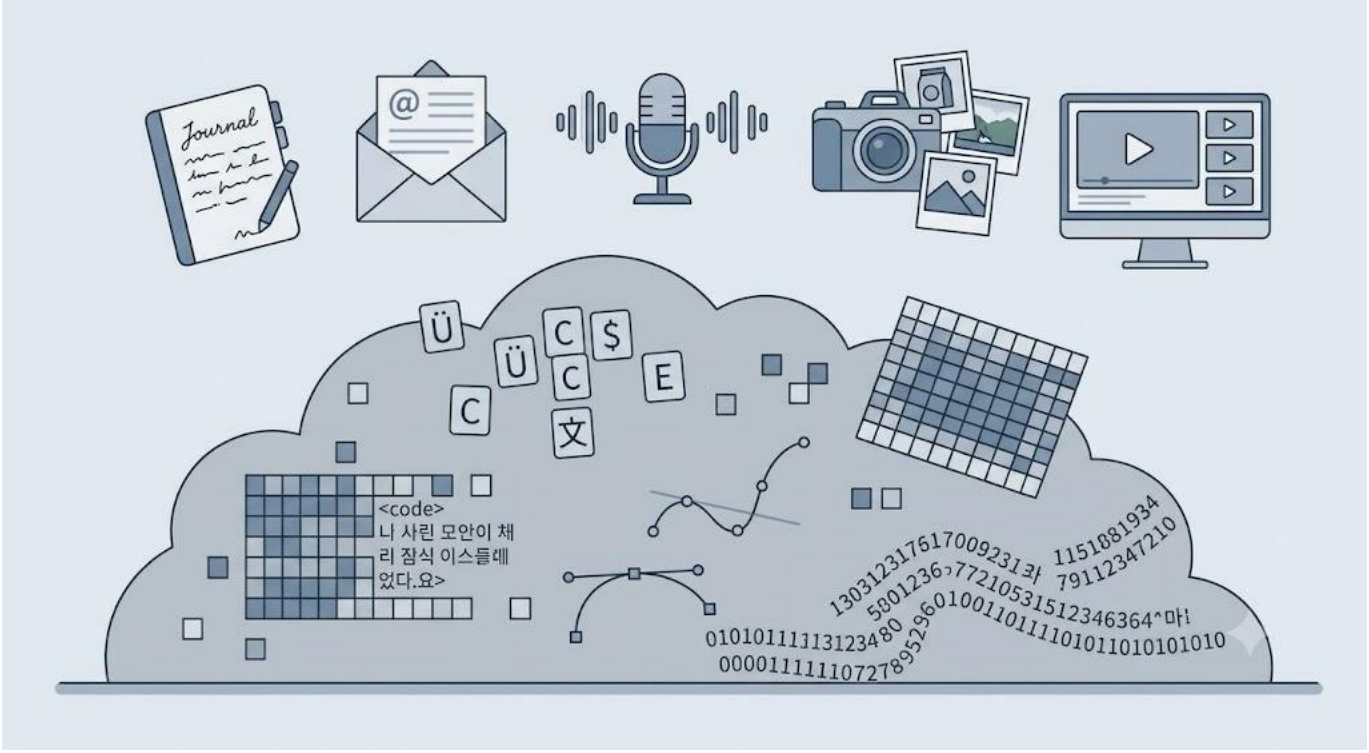


	나이	가격	입사일	회원 번호
회원 1	28	₩ 1,200,000	2021-12-11	M001
회원 2	20	₩ 850,000	2021-02-11	M002
회원 3	35	₩ 980,000	2022-03-29	M003
회원 4	22	₩ 551,000	2022-02-28	M004
회원 5	41	₩ 1,200,000	2022-07-23	M005

정형 데이터와 비정형 데이터

◉ 비정형 데이터 (Unstructured Data)

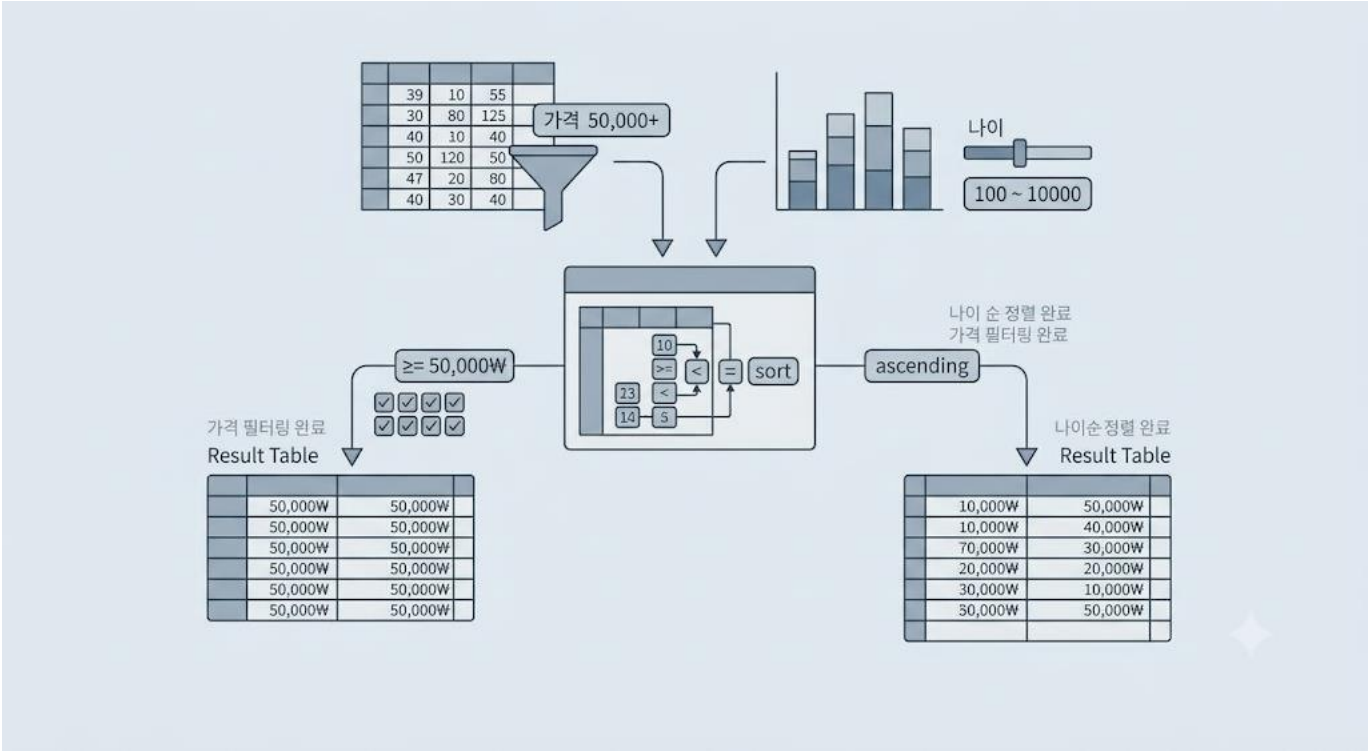
- 일기장, 이메일 본문, 고객 상담 통화 녹음, 제품 사진, 유튜브 영상처럼 정해진 칸에 깔끔하게 넣기 어려운 형태입니다.
- 데이터 안에 나름의 문맥이나 픽셀 구성은 존재하지만, 컴퓨터가 즉시 읽고 연산할 수 있는 통일된 규격이나 좌표가 없습니다.



컴퓨터가 데이터를 다루는 방식

◉ 정형 데이터

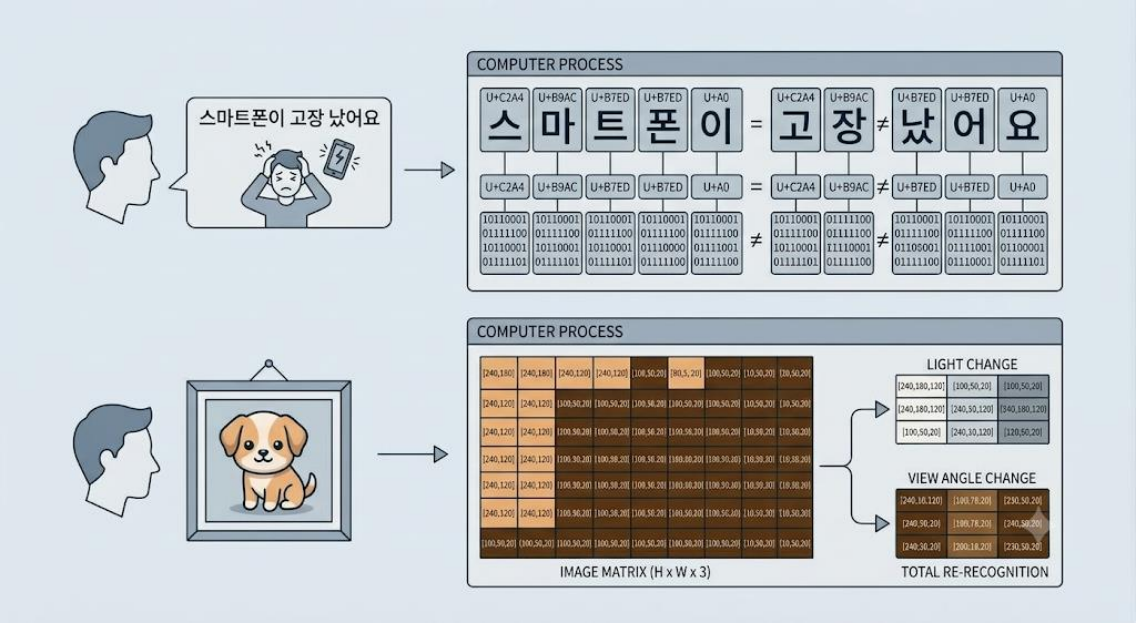
- 계산과 비교가 매우 쉬움
- 컴퓨터는 숫자를 크다, 작다, 같다로 비교하는 연산에 최적화되어 있습니다.
- 엑셀에서 "가격이 50,000원 이상인 것만 필터링"하거나 "나이 순으로 정렬"하는 작업은 컴퓨터 입장에서 단순한 숫자 대소 비교라 수 밀리초 안에 끝납니다.



컴퓨터가 데이터를 다루는 방식

◉ 비정형 데이터

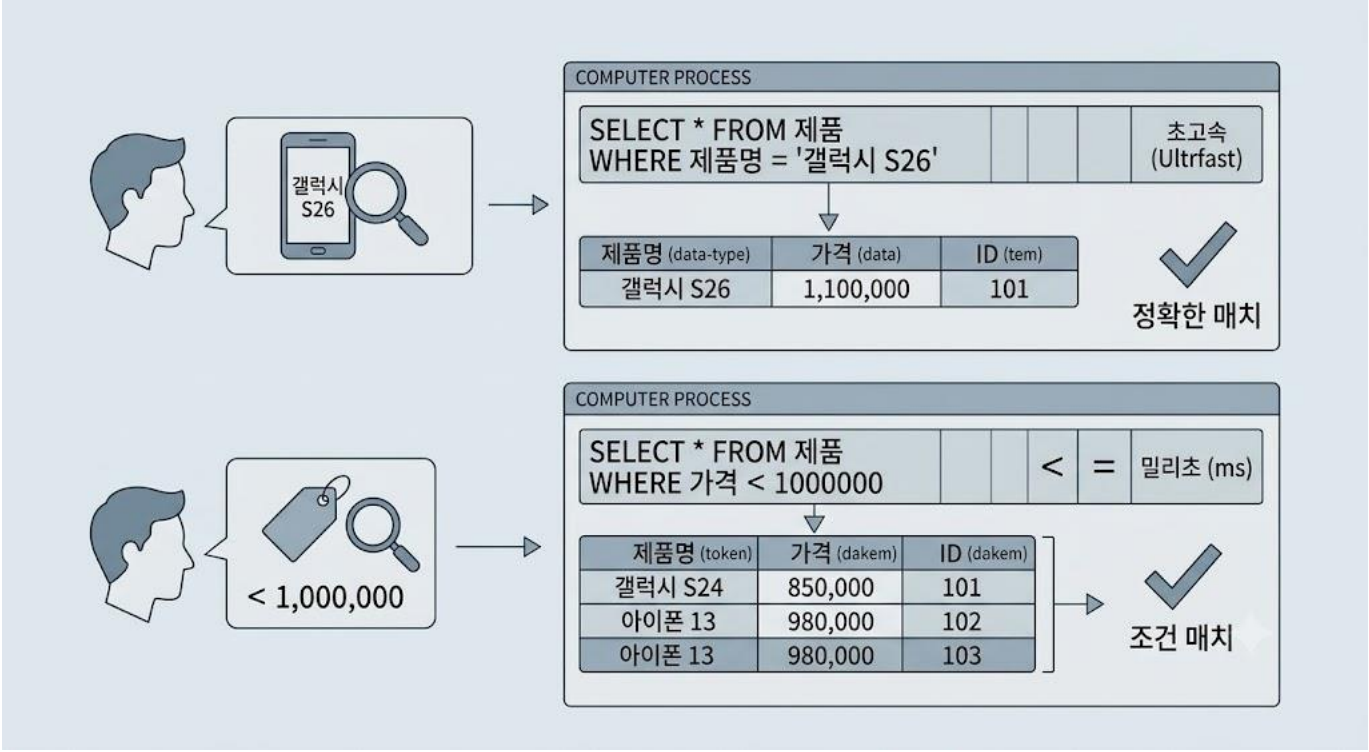
- 컴퓨터 눈에는 의미 없는 기호나 숫자의 나열
- 텍스트: "스마트폰이 고장 났어요"라는 문장은 컴퓨터에게 한글 유니코드 바이트(글자 코드)의 나열일 뿐, 기계가 망가져 곤란해하는 '상황'이라는 의미를 스스로 이해하지 못합니다.
- 이미지: 귀여운 강아지 사진을 컴퓨터에 주면, 컴퓨터는 그저 가로×세로 격자 위에 놓인 빨강/초록/파랑(RGB) 색상 숫자(0~255)들의 배열로만 바라봅니다. 조명이 어두워지거나 카메라 각도만 살짝 바뀌어도 컴퓨터 입장에서는 완전히 다른 데이터로 인식됩니다.



검색할 때 발생하는 결정적 한계

◉ 정형 데이터의 검색

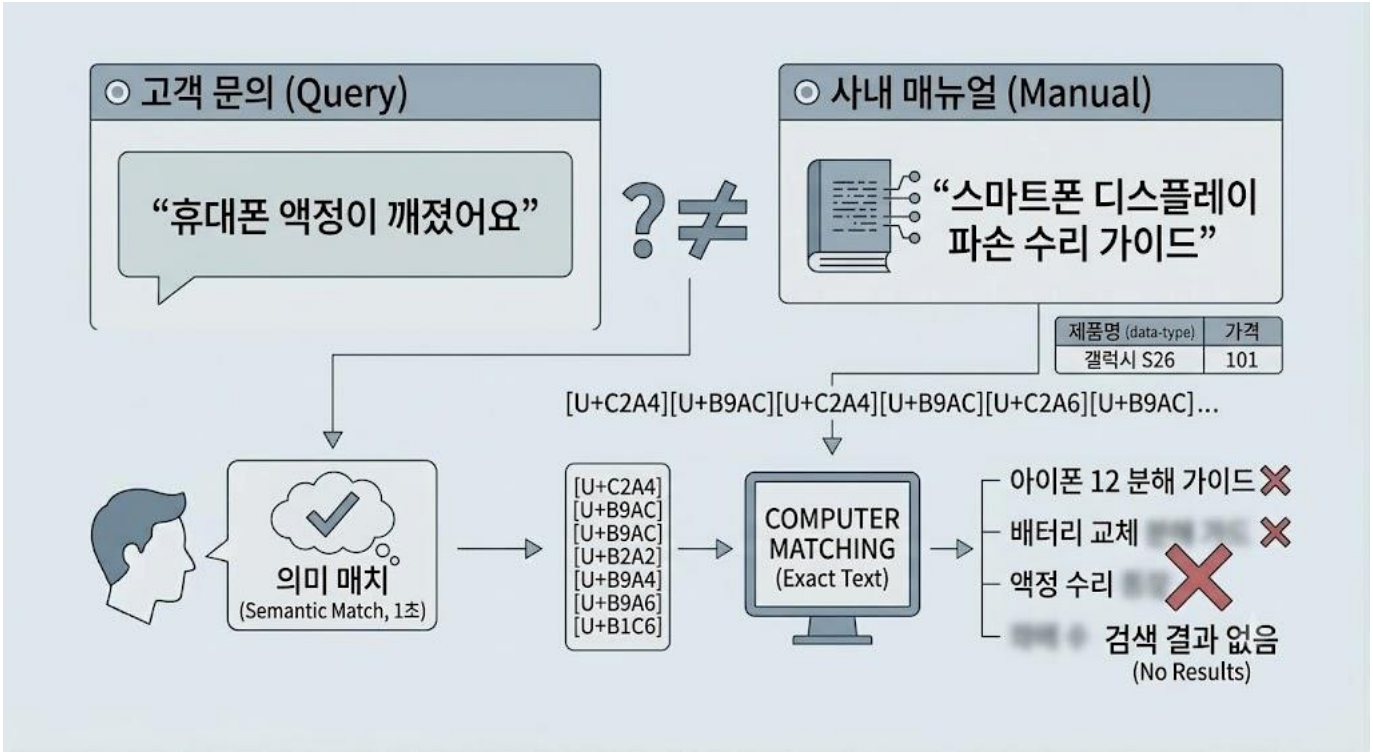
- 정확하고 빠름
- 데이터베이스에서 WHERE 제품명 = '갤럭시 S26' 또는 WHERE 가격 < 1000000과 같이 명확한 조건식을 주면 한 치의 오차도 없이 일치하는 행을 찾아냅니다.



검색할 때 발생하는 결정적 한계

◉ 비정형 데이터의 검색

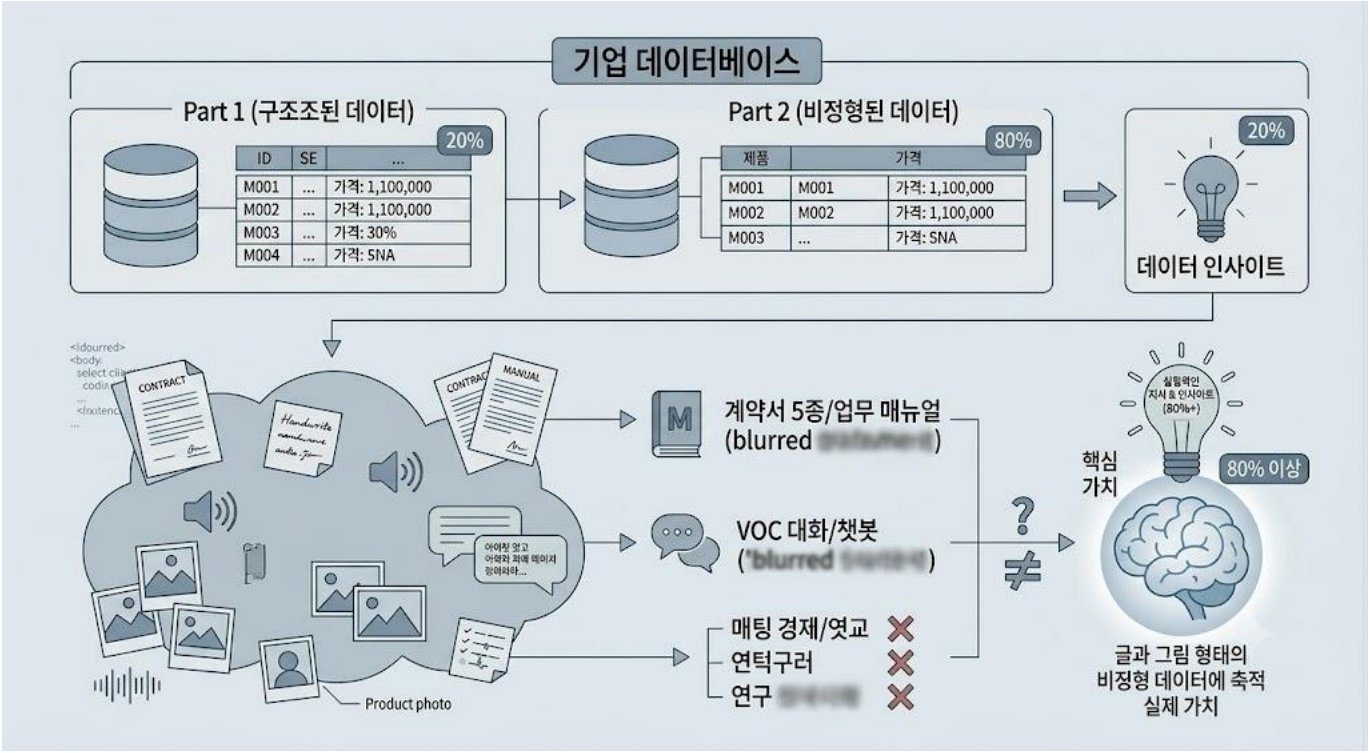
- 키워드가 조금만 달라져도 놓침
- 전통적인 텍스트 검색은 '적힌 글자가 완벽히 같은가?'만 확인합니다.
- 고객이 "휴대폰 액정이 깨졌어요"라고 문의 글을 남겼을 때, 사내 매뉴얼에 "스마트폰 디스플레이 파손 수리 가이드"라고 적혀 있다면 글자가 하나도 겹치지 않아 검색 결과에 나타나지 않습니다.
- 사람이 보면 두 문장이 같은 의미라는 것을 1초 만에 알 수 있지만, 컴퓨터는 글자 모양만 비교하기 때문에 찾지 못하는 것입니다.



왜 지금 '비정형 데이터'가 화두인가?

◉ 데이터 비중의 변화

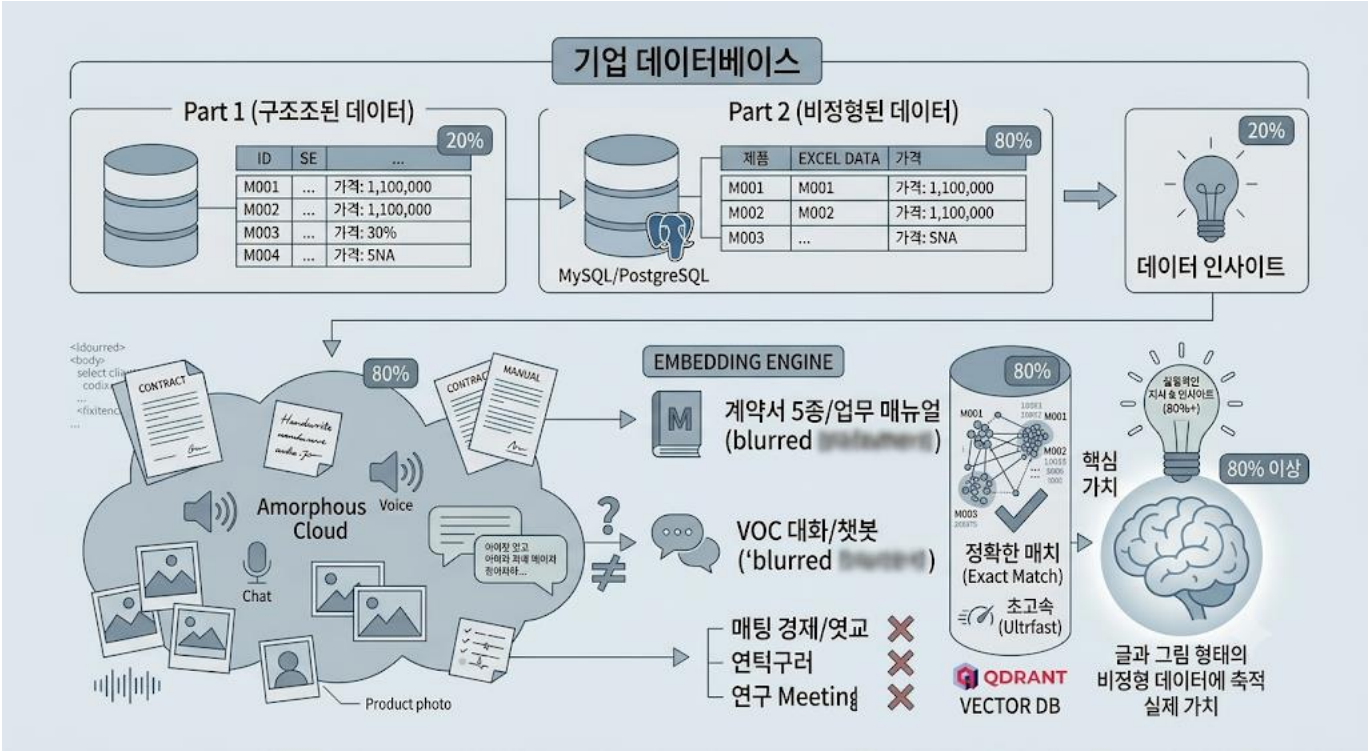
- 실제 기업이 보유한 데이터 중 엑셀이나 DB에 깔끔하게 정리된 정형 데이터는 약 20%에 불과합니다.
- 계약서, 회의록, 업무 매뉴얼, 고객 VOC, 상담 챗봇 대화 등 실질적인 지식과 인사이트의 80% 이상은 모두 '글과 그림' 형태의 비정형 데이터로 쌓여 있습니다.



왜 지금 '비정형 데이터'가 화두인가?

◉ **Vector DB와 임베딩이 필요한 이유**

- 엑셀 표는 기존 관계형 데이터베이스(MySQL, PostgreSQL 등)로도 충분히 완벽하게 다룰 수 있습니다.
- 하지만 "글과 그림 속에 담긴 의미를 파악해서 검색하고 싶다"는 요구를 해결하기 위해서는, 컴퓨터가 이해하지 못하는 비정형 데이터를 '의미가 비슷한 것끼리 가까이 모이도록 숫자로 변환(임베딩)'하여 저장해 줄 Vector DB(Qdrant)가 반드시 필요해진 것입니다.





1. 비정형 데이터와 텍스트 수치화 기법

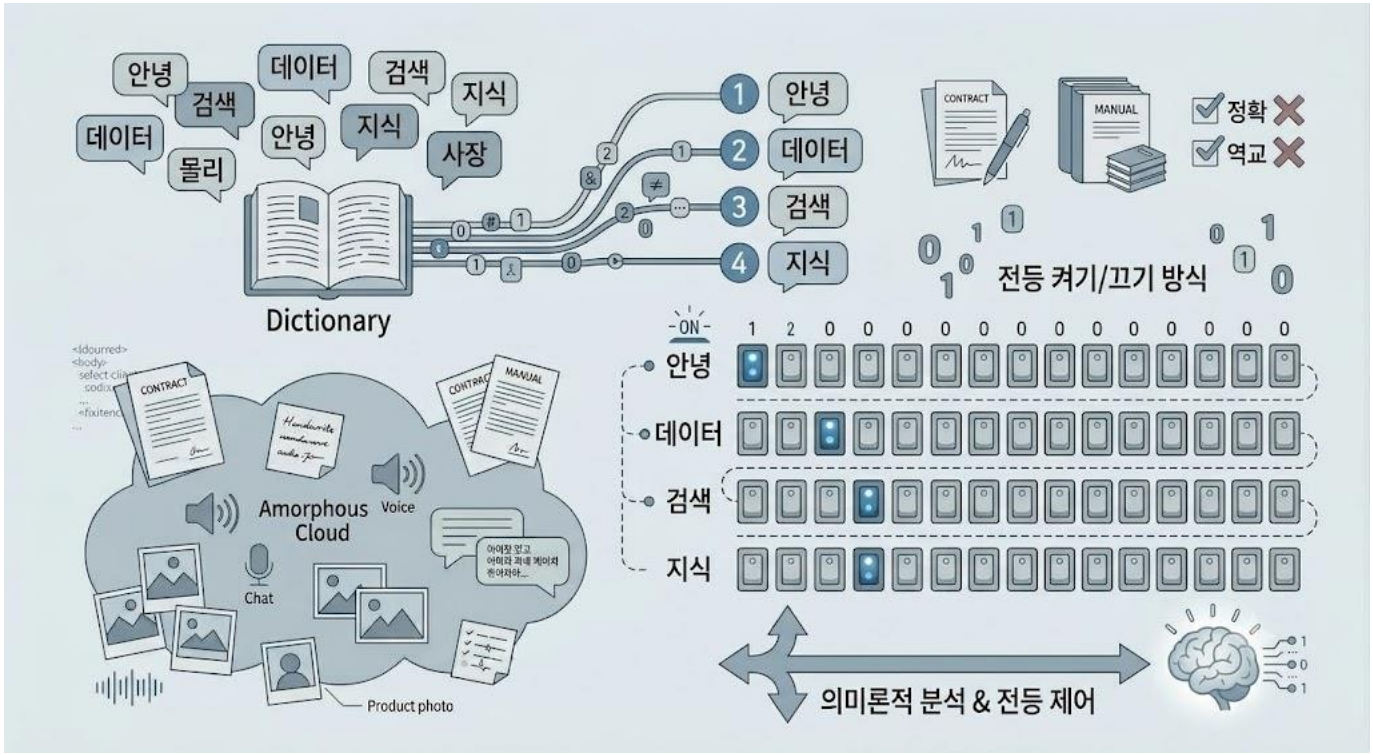
- 전통적인 텍스트 수치화 기법

SK 주식회사 AX

원-핫 인코딩(One-Hot Encoding)이란?

◉ 동작 원리

- 단어마다 고유한 방 번호표 주기
- 다루어야 할 모든 단어를 모아 단어 사전(Dictionary)을 만듭니다.
- 각 단어에 중복되지 않는 고유한 번호(인덱스)를 1번부터 차례대로 부여합니다.
- 단어 하나를 표현할 때, 전체 사전 크기만큼의 전등(스위치)을 일렬로 늘어놓고 자기 방 번호 위치의 전등만 1(ON)로 켜고, 나머지 방은 전부 0(OFF)으로 꺼두는 방식입니다.



원-핫 인코딩(One-Hot Encoding)이란?

◉ 직관적인 예시

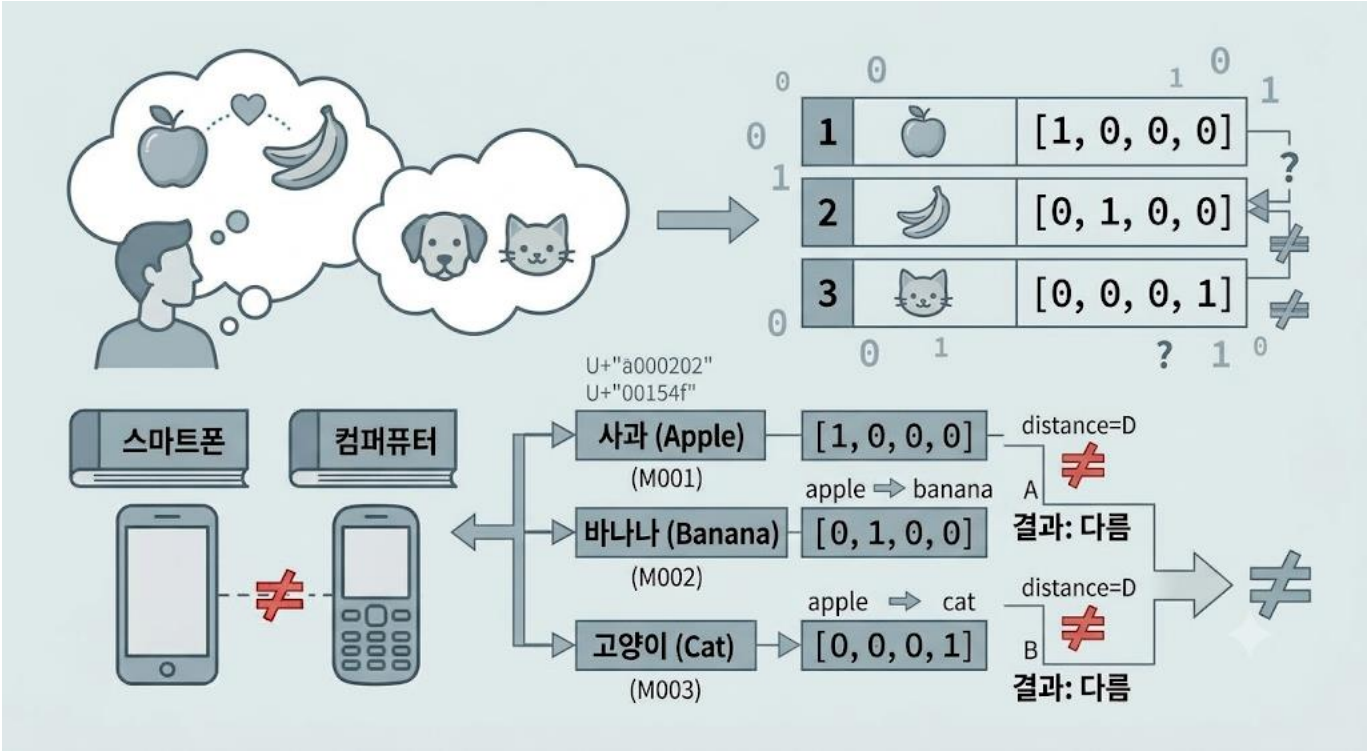
- 우리가 가진 단어가 딱 4개뿐인 세상이라고 가정해 봅니다: ["사과", "바나나", "강아지", "고양이"]



원-핫 인코딩의 치명적인 한계

◉ 단어 간 '의미 관계'를 전혀 알 수 없음

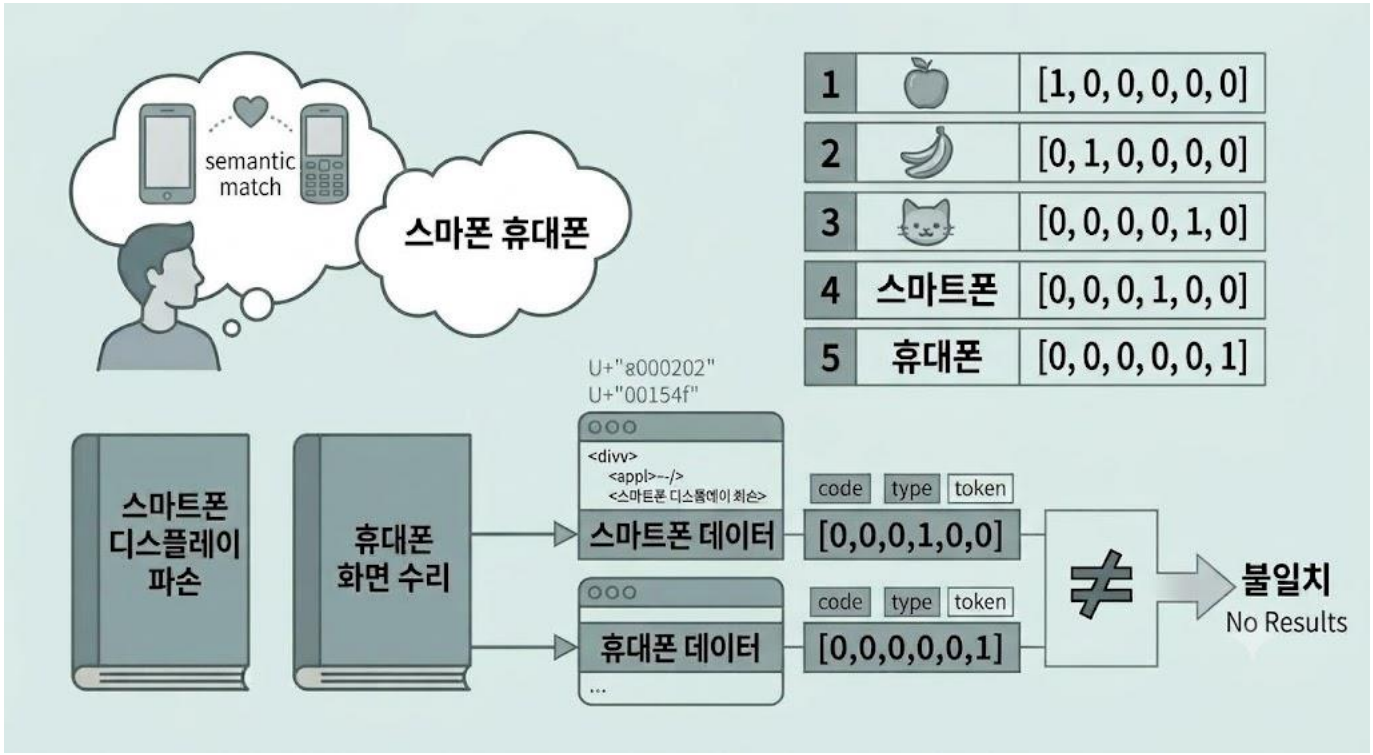
- 단어끼리 얼마나 비슷한지 구분할 방법이 없음
- 사람이 볼 때는 "사과"와 "바나나"는 같은 과일이고, "강아지"와 "고양이"는 같은 동물이라는 느낌이 있습니다.
- 하지만 원-핫 인코딩에서는 사과: [1, 0, 0, 0], 바나나: [0, 1, 0, 0], 고양이: [0, 0, 0, 1]
- 컴퓨터 눈에는 "사과와 바나나"도 그저 서로 다른 방 번호일 뿐이고, "사과와 고양이"도 서로 다른 방 번호일
- 뿐입니다. 즉, 어떤 단어 쌍을 가져와도 '그냥 번호가 다르다'는 사실 외에는 아무런 차이를 읽어내지 못합니다.



원-핫 인코딩의 치명적인 한계

◉ 동의어·유의어 인식 불가

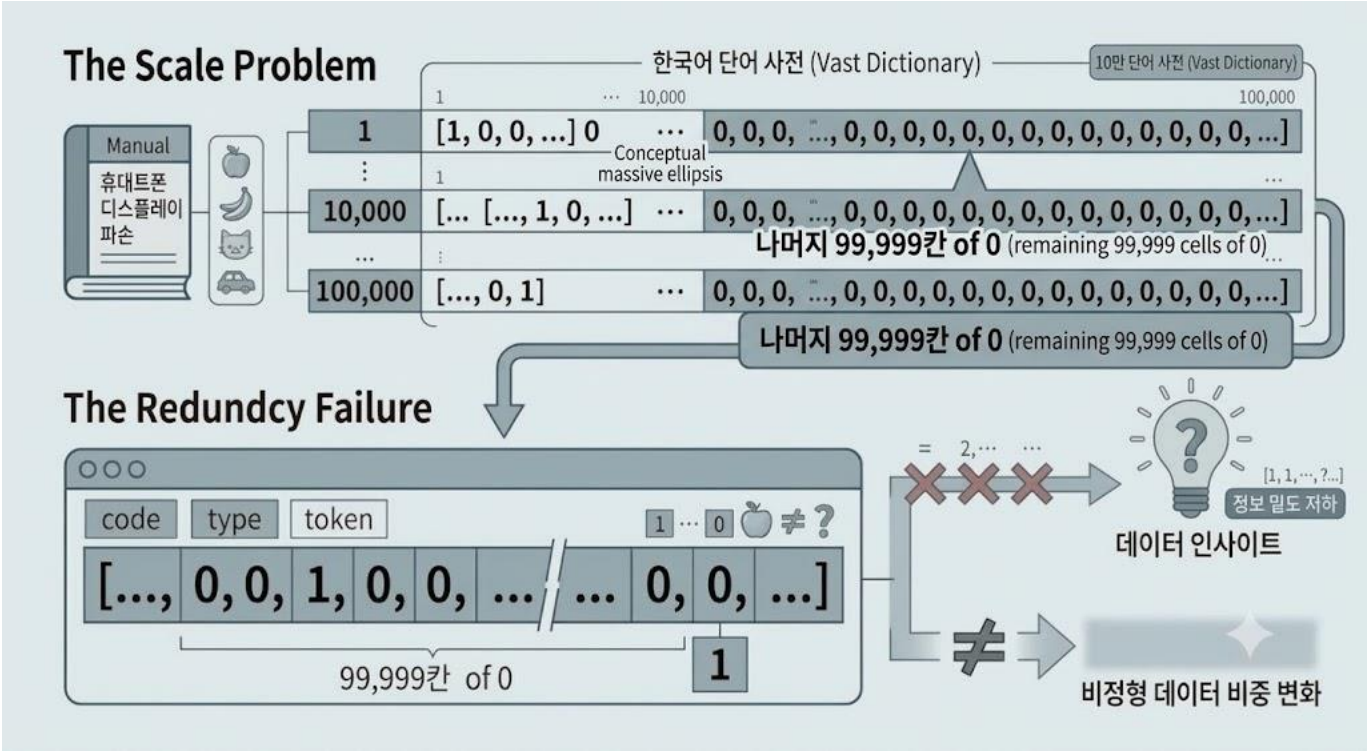
- "스마트폰"과 "휴대폰"은 사실상 같은 뜻이지만, 서로 다른 방 번호를 부여받는 순간 컴퓨터는 둘을 완전히 남남인 별개의 단어로 취급합니다.
- 그 결과, 같은 주제를 이야기하고 있어도 단어 표기가 조금만 다르면 아무런 연관성을 찾지 못합니다.



원-핫 인코딩의 치명적인 한계

◉ 불필요한 0의 폭발:

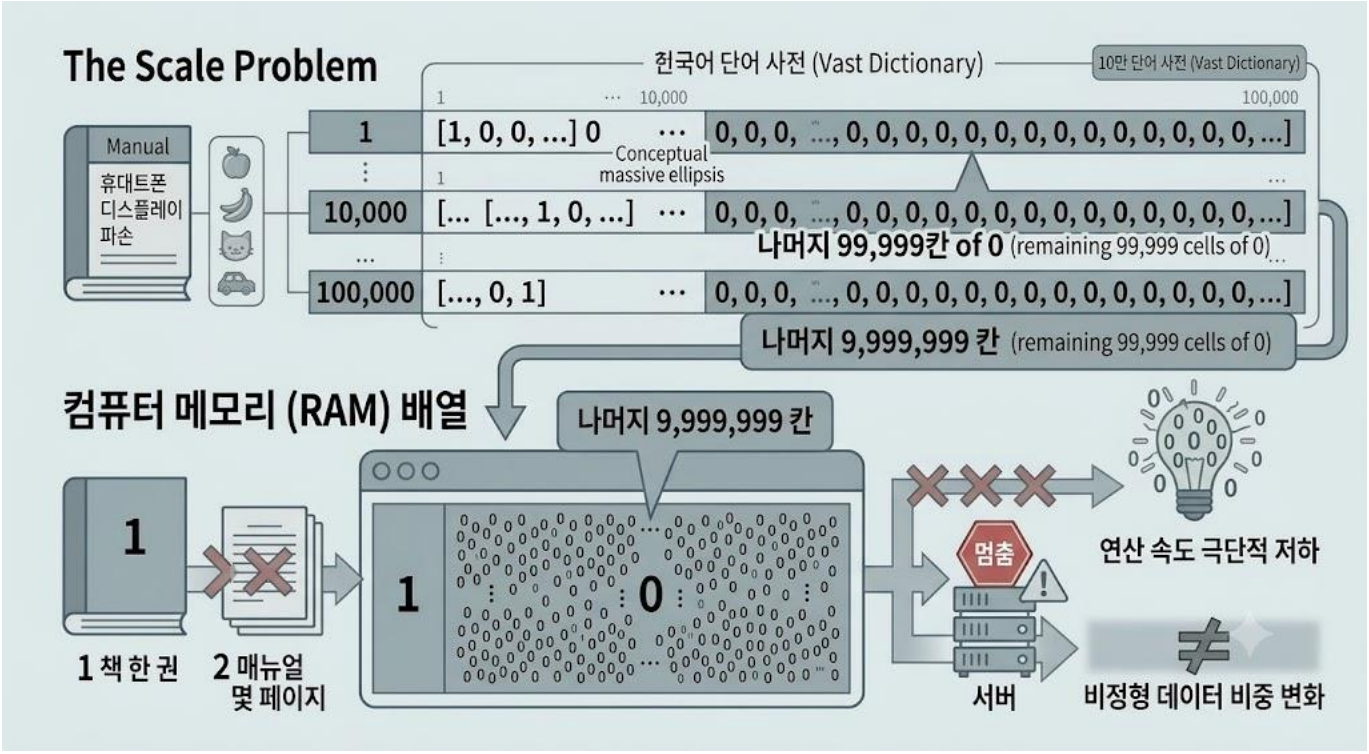
- 일상생활이나 회사 업무에서 쓰이는 한국어 단어는 보통 수만 개에서 수십만 개에 달합니다.
- 단어 사전이 10만 개라면, 단어 하나를 표현하기 위해 10만 칸짜리 배열이 필요합니다.
- 문제는 10만 칸 중에 단 1칸만 1이고, 나머지 99,999칸은 아무 의미 없는 0으로 채워진다는 점입니다.



원-핫 인코딩의 치명적인 한계

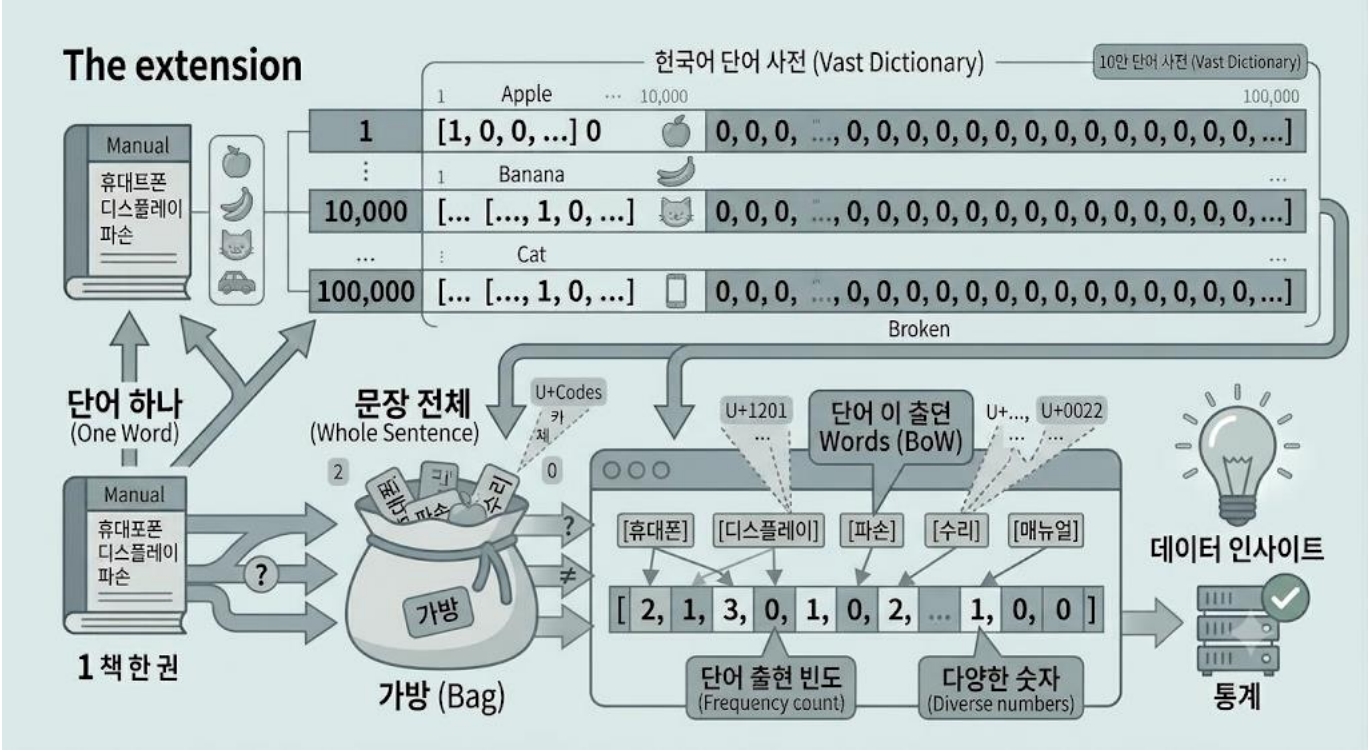
◉ 시스템 과부하:

- 책 한 권, 매뉴얼 몇 페이지만 숫자로 바꿔도 컴퓨터 메모리(RAM)가 0으로 가득 찬 거대한 배열들로 채워져 서버가 멈추거나 연산 속도가 극단적으로 느려집니다.



빈도수 기반 카운팅(Bag of Words / CountVectorizer)

◉ 원-핫 인코딩이 "단어 하나"에 번호표를 붙였다면, Bag of Words(BoW)는 이를 확장해 "문장 전체"를 하나의 숫자 배열로 바꾸는 전통적인 통계 기반 방식입니다.



빈도수 기반 카운팅(Bag of Words / CountVectorizer)

◉ 동작 원리

- CountVectorizer (카운트 벡터라이저)
- 전체 문서에 등장하는 단어들을 모아 중복 없는 단어 사전을 만듭니다.
- 각 문장을 읽으면서 사전에 있는 단어가 몇 번 나왔는지 숫자를 카운트합니다.
- 그 카운트 숫자를 단어 사전 순서대로 나열하여 하나의 벡터(숫자 배열)를 만듭니다.

쉬운 예시로 이해하기

단어 (Word)	U+ Code	사과 🍎	바나나 🍌	먹었다 👧	맛있다 🐶	안
사과	U+0001	1	0	0	0	0
바나나	U+0002	0	1	0	0	0
먹었다	U+0003	0	0	1	0	0
맛있다	U+0004	0	0	0	1	0
안	U+0005	0	0	0	0	1



빈도수 기반 카운팅의 치명적인 한계

◉ 어순(문맥)을 완전히 무시함

- 가방에 넣고 흔들어버렸기 때문에, 단어의 앞뒤 순서가 사라집니다.
- 예시:
  - 문장 A: "나는 밥을 먹었다"
  - 문장 B: "밥이 나를 먹었다"
- 사람은 두 문장이 완전히 다른 뜻(하나는 공포 영화)이라는 것을 알지만, Bag of Words는 두 문장 모두 {"나": 1, "밥": 1, "먹었다": 1}로 똑같이 세어버려 컴퓨터 입장에서는 100% 동일한 문장으로 취급됩니다.
- 부정어 처리의 왜곡:
  - "이 식당은 맛있다 안 친절하다"
  - "이 식당은 안 맛있다 친절하다"
- 단어 출현 횟수가 같으므로 컴퓨터는 두 평가를 구분하지 못합니다.

빈도수 기반 카운팅의 치명적인 한계

◉ 의미 없는 단어가 가장 중요하게 취급됨 (빈도의 함정)

- 문장에서 가장 자주 등장하는 단어는 보통 의미가 풍부한 단어가 아니라 문법을 맞추기 위한 조사나 불용어입니다.
- 예: "은/는/이/가", "그리고", "하지만", "있다", "the", "a"
- 단순히 많이 나온 단어에 높은 숫자가 찍히다 보니, 정작 중요한 핵심 키워드(예: "데이터베이스", "오류")보다 조사나 접속사가 문장의 대표 특징으로 둔갑해 버립니다.

빈도수 기반 카운팅의 치명적인 한계

◉ 원-핫 인코딩의 '메모리 낭비'와 '동의어 무시' 문제를 그대로 상속

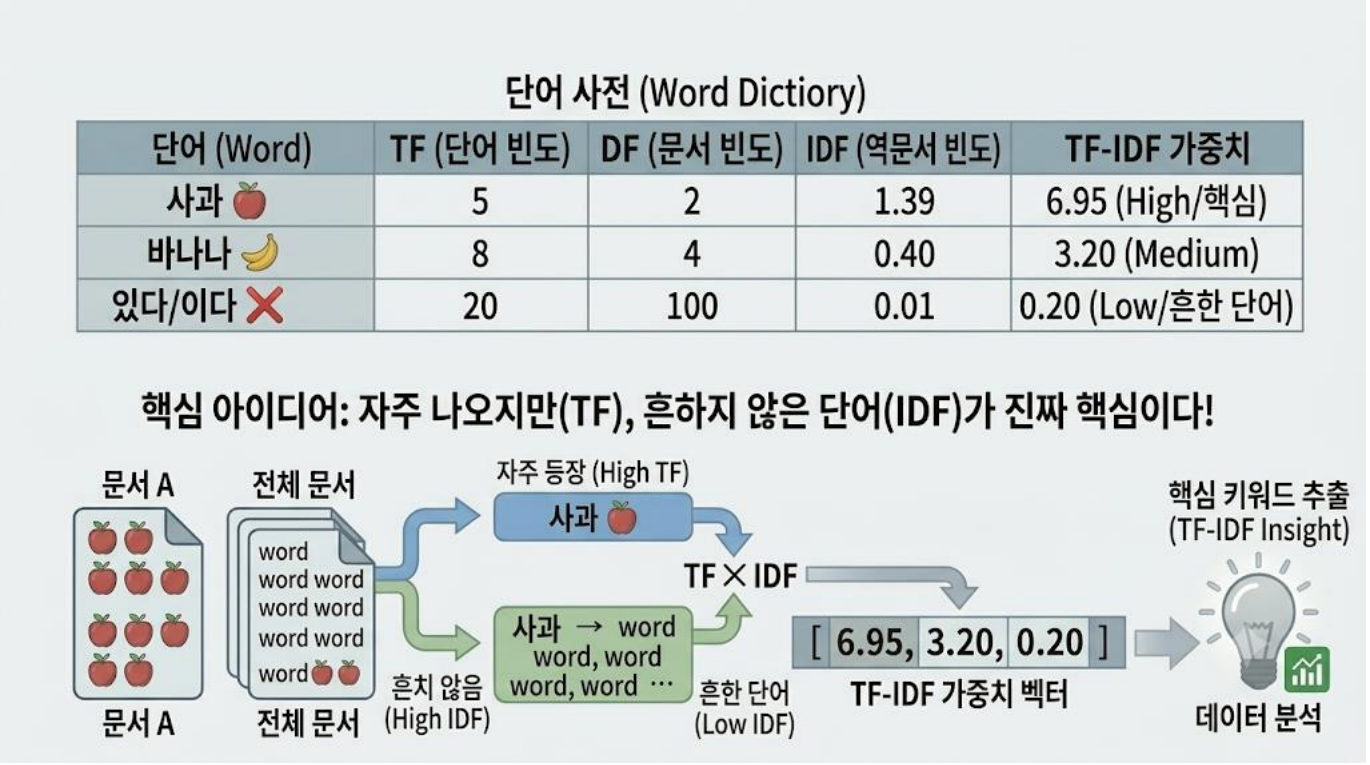
- 문서 전체 단어 수가 5만 개라면, 고작 세 단어로 된 짧은 문장 하나도 5만 칸짜리 배열로 만들어야 합니다 (대부분의 칸이 0).
- "휴대폰을 샀다"와 "스마트폰을 샀다"는 쓰는 단어가 다르기 때문에 겹치는 카운트가 없어 완전히 다른 문장으로 인식됩니다.

TF-IDF

◉ Bag of Words가 가진 "무조건 많이 나오면 중요한 단어인가?"라는 허점을 보완하기 위해 고안된 통계 기반 가중치 기법입니다.

◉ 핵심 아이디어: "자주 나오지만(TF), 흔하지 않은 단어(IDF)가 진짜 핵심이다!"

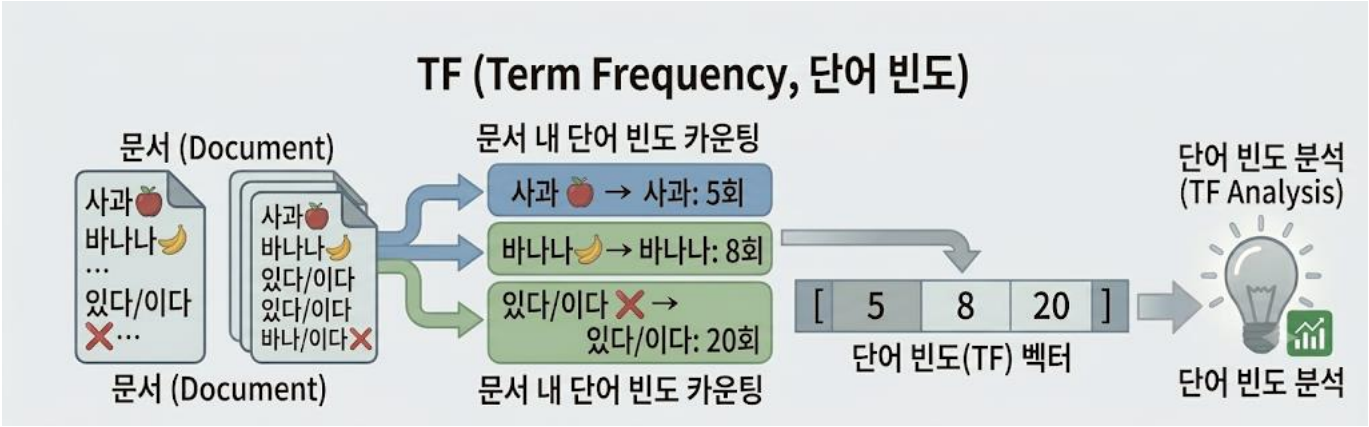
- 어떤 단어가 현재 읽고 있는 문서 안에서는 엄청 자주 나오는데, 다른 모든 문서들에서는 거의 보이지 않는다면 그 단어가 바로 이 문서를 대표하는 진짜 알짜배기 핵심 키워드라는 원리입니다.



TF-IDF

◉ TF (Term Frequency, 단어 빈도): "이 글 안에서 얼마나 자주 나왔는가?"

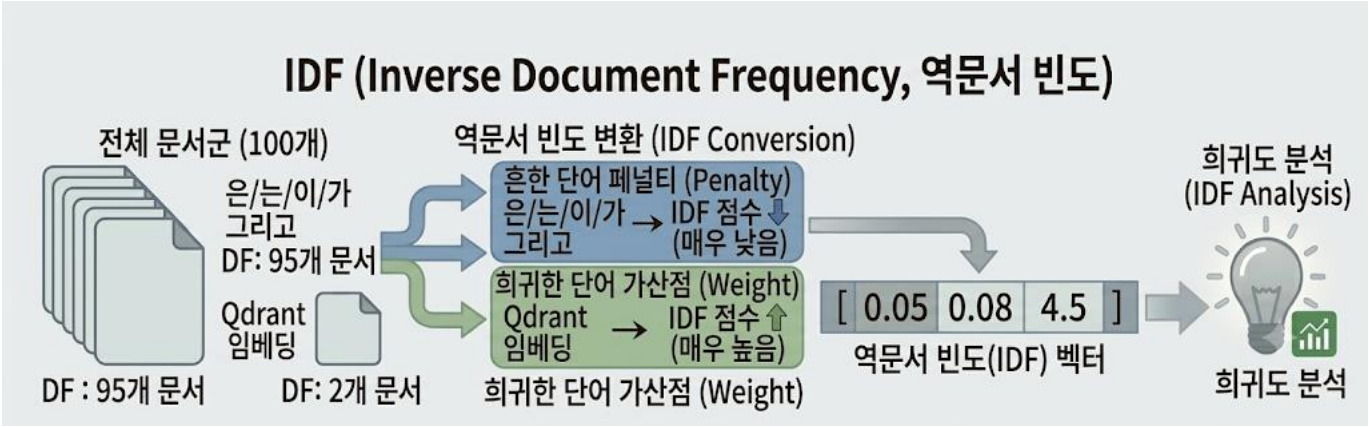
- 특정 문서 안에서 그 단어가 등장하는 횟수입니다.
- 당연히 글 안에서 여러 번 언급될수록 그 글의 주제와 관련 있을 확률이 높습니다.



TF-IDF

◉ IDF (Inverse Document Frequency, 역문서 빈도): "다른 문서들까지 다 뒤졌을 때 얼마나 희귀한가?"

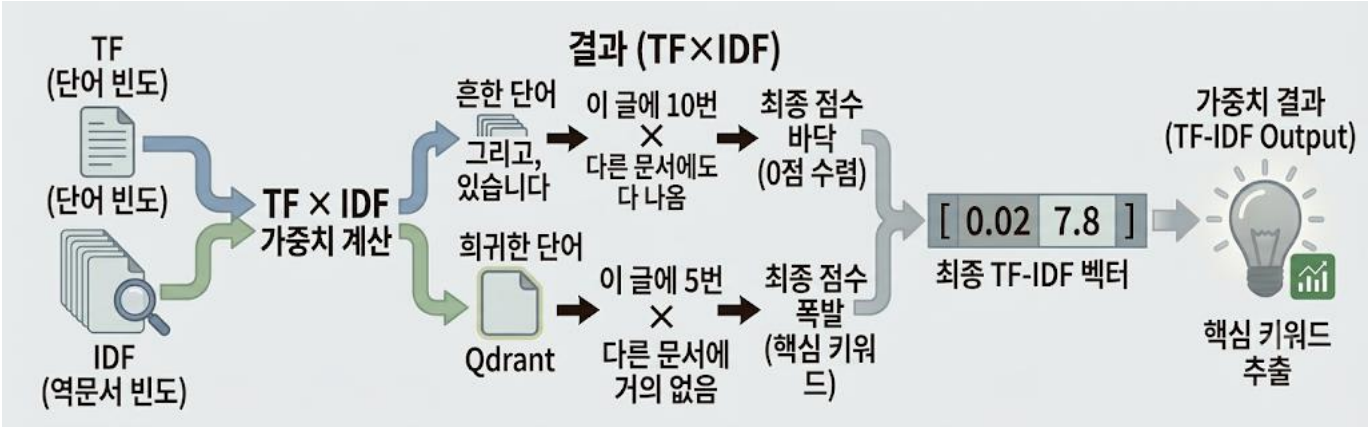
- '역(Inverse)'이라는 말은 '반대로 뒤집었다'는 뜻입니다.
- 모든 글에서 다 나오는 흔해 빠진 단어(예: "은/는/이/가", "그리고", "기능", "제품")는 점수를 대폭 깎아버립니다(페널티).
- 반대로 100개의 문서 중 딱 1~2개 문서에만 등장하는 희귀한 단어(예: "Qdrant", "냉매누설", "임베딩")는 높은 가산점을 줍니다.



TF-IDF

◉ 결과 (TF×IDF)

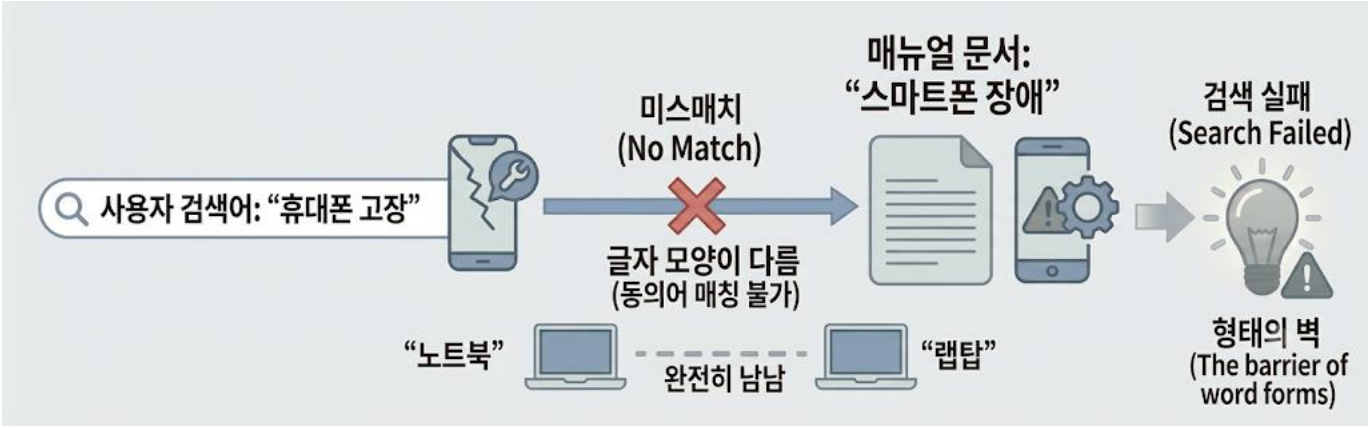
- (이 글에서의 등장 횟수) × (전체 문서에서의 희귀도)
- "그리고", "있습니다"→이 글에 10번 나와도, 다른 글에도 다 나오니까 최종 점수 바닥 (0점에 수렴)
- "Qdrant"→이 글에 5번 나왔고, 다른 글에는 거의 안 나오니까 최종 점수 폭발 (핵심 키워드로 등극)



TF-IDF의 한계

◉ 동의어와 유의어를 여전히 모름 (형태의 벽)

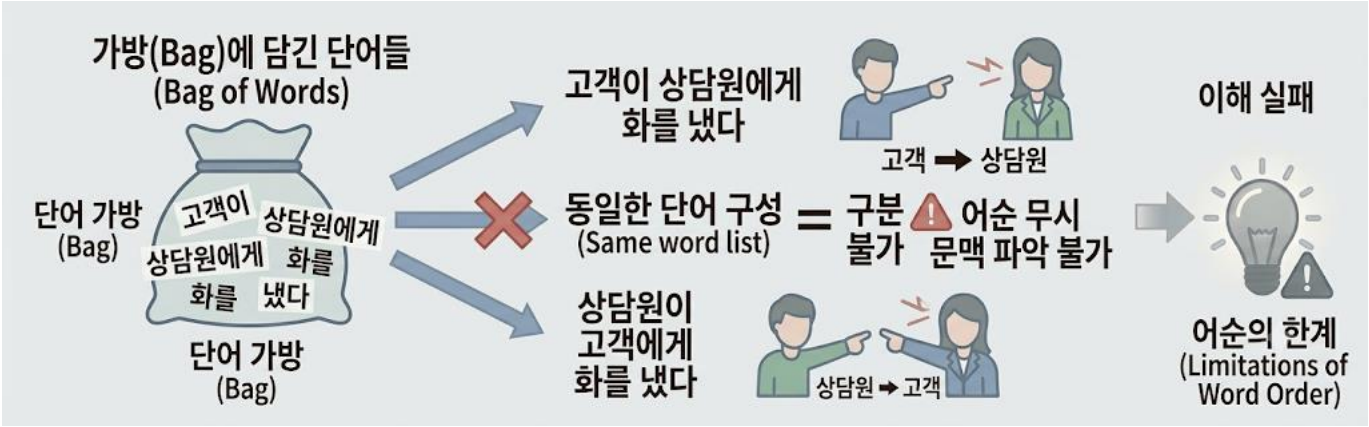
- 단어의 글자 모양만 따지기 때문에 "스마트폰"과 "휴대폰", "노트북"과 "랩탑"을 여전히 완전히 다른 단어로 취급합니다
- 사용자가 "휴대폰 고장"으로 검색했을 때, 매뉴얼에 "스마트폰 장애"라고만 적혀 있다면 아무리 TF-IDF 점수가 높아도 매칭되지 않습니다.



TF-IDF의 한계

◉ 어순과 문맥 파악 불가

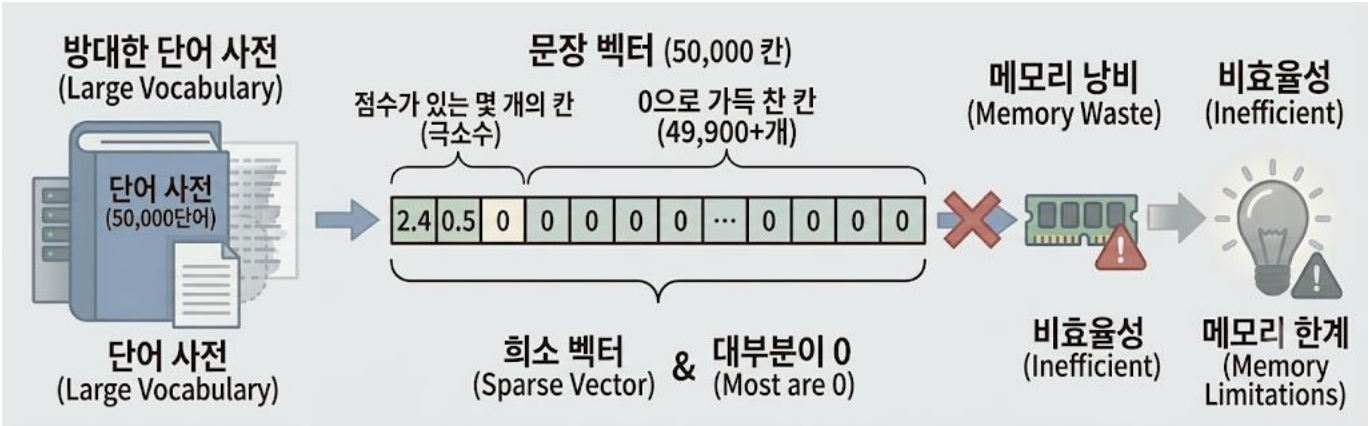
- 단어에 점수를 매겨 가방(Bag)에 담았을 뿐, 여전히 단어의 순서는 보지 못합니다.
- "고객이 상담원에게 화를 냈다"와 "상담원이 고객에게 화를 냈다"는 단어 구성과 희귀도가 같기 때문에 두 문장의 차이를 여전히 구분하지 못합니다.



TF-IDF의 한계

◉ 사전 크기만큼 거대해지는 메모리 문제 (희소성)

- 전체 단어가 5만 개라면, TF-IDF로 만들어지는 문장 벡터도 여전히 5만 칸짜리 배열입니다.
- 몇 개 단어 위치에만 점수(소수점)가 적혀 있고 나머지 49,900여 개의 칸은 여전히 0으로 가득 찬 메모리 낭비 구조입니다.





1. 비정형 데이터와 텍스트 수치화 기법

- 전통 기법의 결정적 한계

SK 주식회사 AX

전통 기법의 결정적 한계

◉ 전통적인 텍스트 수치화 기법(원-핫 인코딩, Bag of Words, TF-IDF, 형태소 분석 기반 역색인)은 모두 "문자열이 눈에 보이게 일치하는가?"를 따지는 기법입니다.

◉ 따라서 단어의 철자(형태)가 조금이라도 달라지면, 사람이 볼 때 완벽히 같은 뜻이라도 컴퓨터는 전혀 상관없는 별개의 데이터로 취급합니다.



전통 기법의 결정적 한계

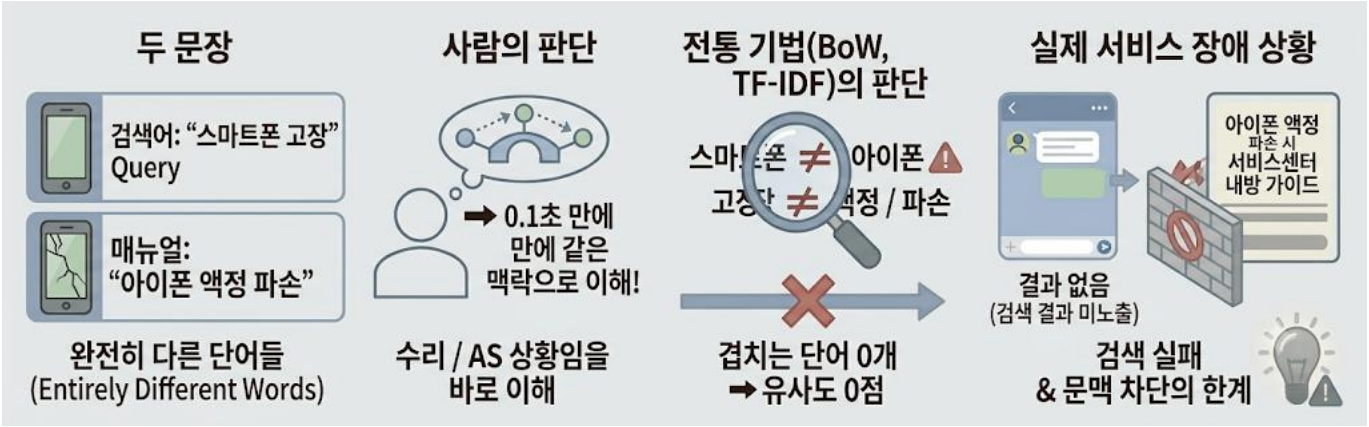
◉ 스마트폰 고장" vs "아이폰 액정 파손"사람의 판단

- "아, 아이폰 화면이 깨져서 수리나 AS가 필요한 상황이구나!" 하고 0.1초 만에 같은 맥락으로 이해합니다.

◉ 전통 기법(TF-IDF 등)의 판단

- 단어 비교:스마트폰≠아이폰, 고장≠액정, 고장≠파손
- 겹치는 단어(교집합): 0개
- 계산된 유사도 점수: 0점 (완전히 무관한 문서로 판정)

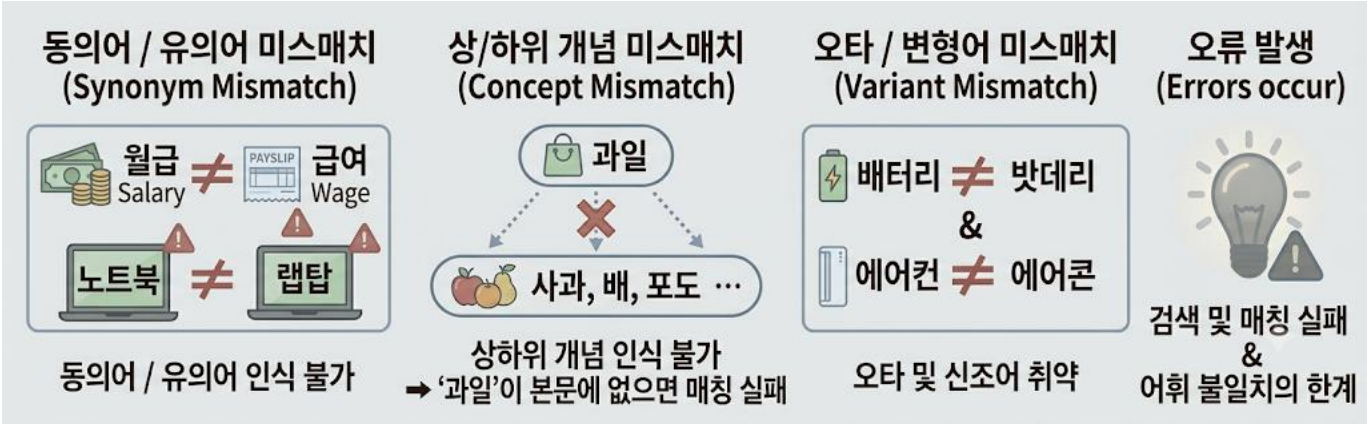
◉ 실제 서비스에서 발생하는 사고:고객이 챗봇 검색창에 "스마트폰 고장 났을 때 접수 방법"이라고 검색했는데, 사내 매뉴얼 제목이 "아이폰 액정 파손 시 서비스센터 내방 가이드"로 되어 있으면 검색 결과에 아예 노출되지 않습니다.



전통 기법의 결정적 한계

◉ 어휘 불일치(Vocabulary Mismatch)가 만드는 일상적 오류들

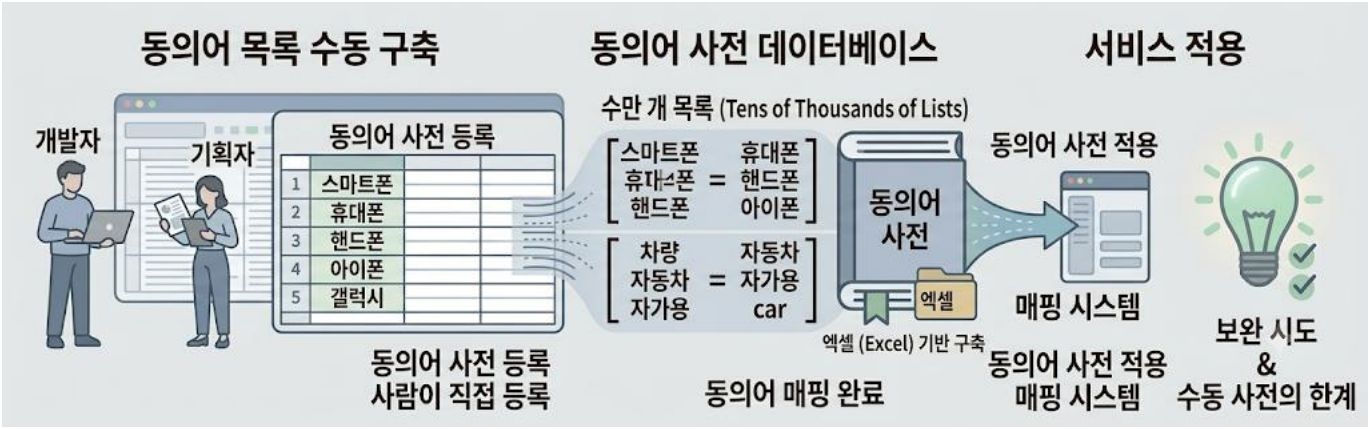
- 동의어 / 유의어 인식 불가
  - "월급"과 "급여", "차량"과 "자동차", "노트북"과 "랩탑"을 완전히 다른 단어로 인식합니다.
- 상위 개념과 하위 개념 인식 불가
  - "과일"을 검색하면 사과, 배, 포도가 나와야 하지만, 문서에 '과일'이라는 단어가 직접 적혀 있지 않으면 사과 문서를 찾아내지 못합니다.
- 오타 및 신조어에 극도로 취약
  - "배터리"와 "밧데리", "에어컨"과 "에어콘"처럼 일상적인 표기 차이만 발생해도 검색 매칭에 실패합니다.



이 한계를 극복한 방법

◉ 동의어 사전(Synonym Dictionary) 구축

- 개발자와 기획자가 엑셀에 ["스마트폰", "휴대폰", "핸드폰", "아이폰", "갤럭시", ...] 같은 동의어 목록을 수동으로 수만 개씩 등록했습니다.



한계 극복이 실패한 이유

◉ 수작업 방식이 실패한 이유

- 끝없는 유지보수: 새로운 신조어, 제품명, 줄임말이 매일 쏟아져 나와 사전 관리가 불가능해집니다.
- 문맥 파악 불가: "사과"의 동의어로 "용서"를 넣으면, "사과 주스"를 검색할 때 "고객에 대한 사과문"이 튀어나오는 대참사가 발생합니다.
- 비용 폭발: 도메인(의료, 법률, 금융, IT)이 바뀔 때마다 전문 용어 사전을 밑바닥부터 새로 만들어야 했습니다.





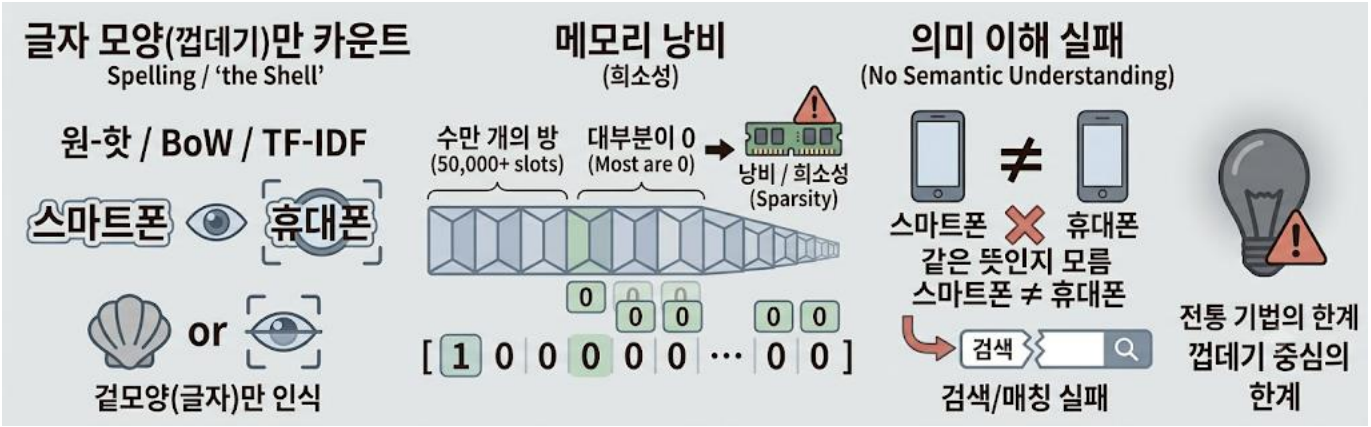
1. 비정형 데이터와 텍스트 수치화 기법

- AI(신경망) 기반 Dense Embedding

SK 주식회사 AX

등장 배경

◉ 전통적인 방식(원-핫, BoW, TF-IDF)은 단어의 '글자 모양(껍데기)'만 세었기 때문에, 사전에 수만 개의 방을 만들어 놓고 대부분을 0으로 채우는 낭비(희소성)가 발생했고, "스마트폰"과 "휴대폰"이 같은 뜻이라는 사실도 알아채지 못했습니다.

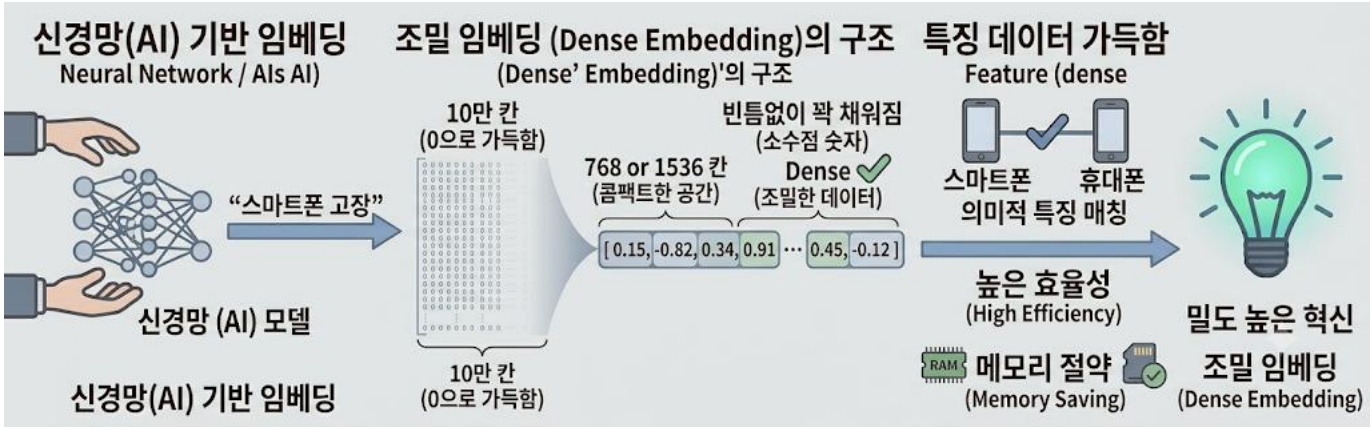


등장 배경

◉ 이 한계를 극복한 기술이 바로 신경망(AI) 기반의 Dense Embedding(조밀 임베딩)입니다.

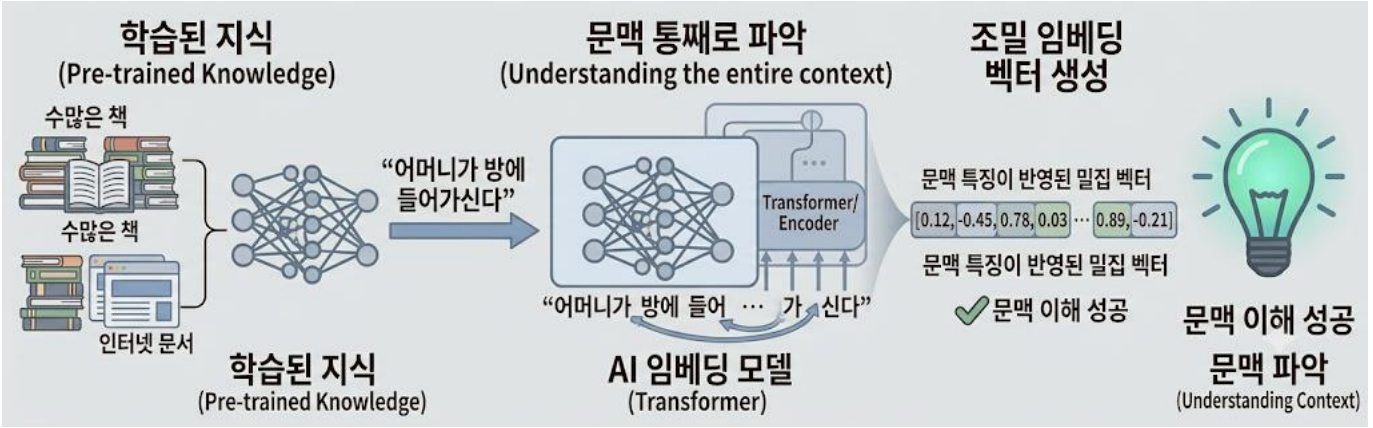
● Dense(조밀하다)의 진짜 의미

- 10만 칸짜리 거대한 방에 0만 가득 채우는 대신, 딱 768칸이나 1536칸 정도의 콤팩트한 공간을 마련합니다.
- 그리고 그 모든 칸을 비워두지 않고, 단어와 문장의 특징을 나타내는 소수점 숫자(연속적인 실수, 예: 0.15, -0.82)들로 빈틈없이 꽉 채워(Dense) 넣는 방식입니다.



의미 좌표로 바꾸는 작업

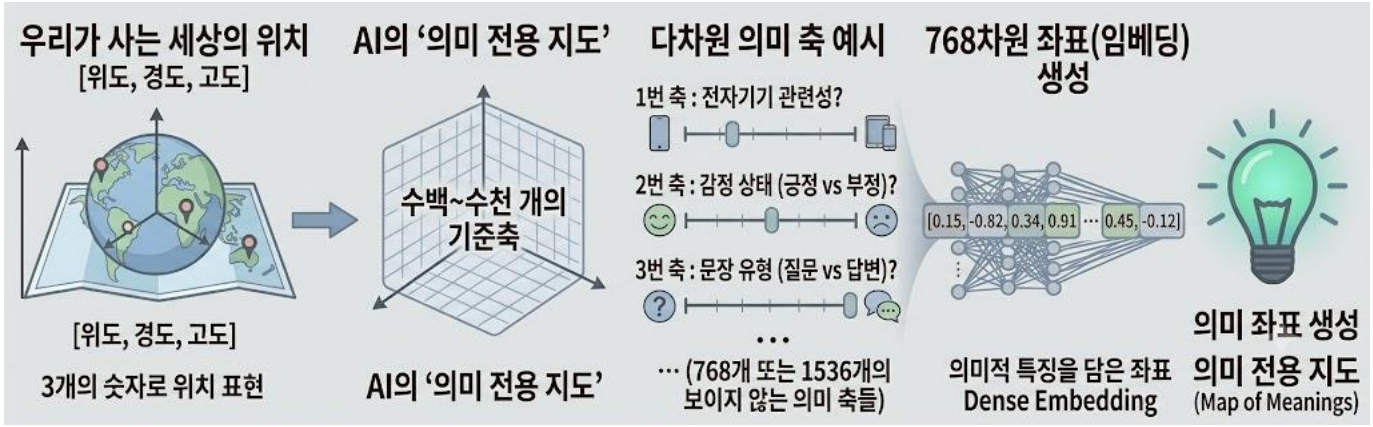
◉ AI 임베딩 모델(Transformer 인코더 등)은 글자를 읽을 때 단순히 단어를 잘라 번호를 붙이지 않습니다. 수많은 책과 인터넷 문서를 읽으며 배운 지식을 바탕으로 문맥을 통째로 파악합니다.



의미 좌표로 바꾸는 작업

◉ 직관적인 비유

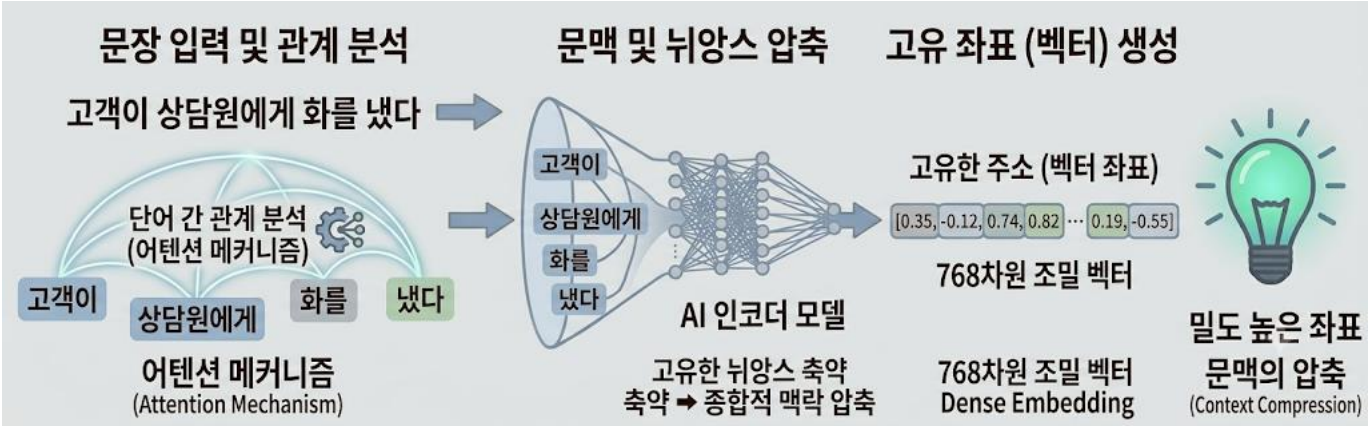
- 우리가 사는 세상의 위치는 [위도, 경도, 고도]라는 3개의 숫자로 표현됩니다.
- AI는 텍스트를 이해하기 위해 수백~수천 개의 기준축을 가진 '의미 전용 지도'를 만듭니다.
- 1번 축: 이것이 전자기기와 관련된 내용인가?
- 2번 축: 긍정적인 감정인가, 부정적인 문제 상황인가?
- 3번 축: 질문인가, 답변 설명인가?
- ... (총 768개 또는 1536개의 보이지 않는 의미 축들)



의미 좌표로 바꾸는 작업

◉ 문맥의 압축

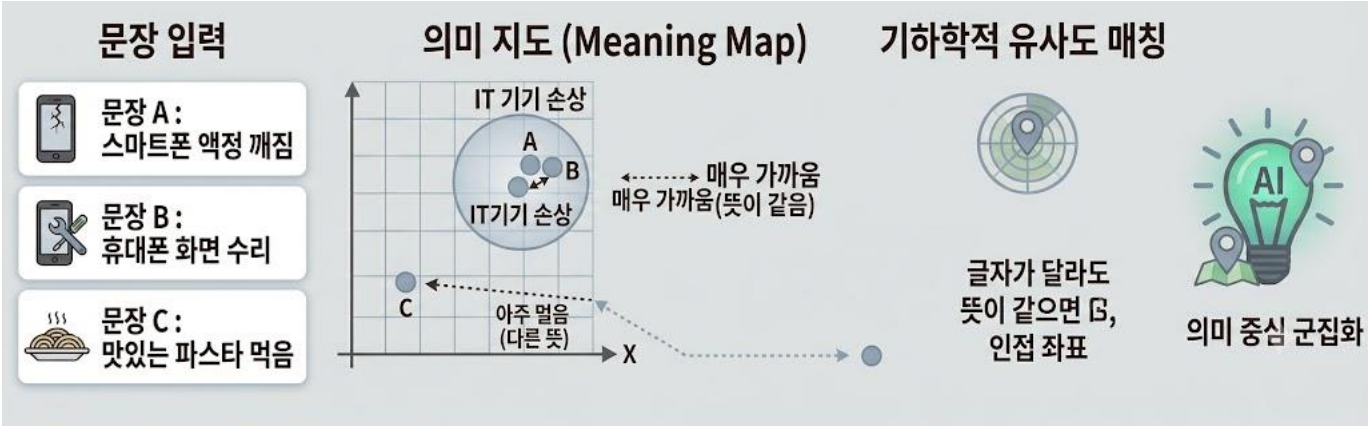
- AI 모델에 문장을 통째로 집어넣으면, 문장 속 단어들이 서로 어떤 관계를 맺고 있는지(어텐션 메커니즘)를 종합적으로 분석합니다.
- 그리고 그 문장이 가진 고유한 뉘앙스를 축약하여 하나의 고유한 주소(실수 숫자들의 묶음, 벡터 좌표)를 만들어줍니다



의미 좌표로 바꾸는 작업

◉ Dense Embedding의 가장 큰 장점은 "글자 모양이 달라도, 뜻이 통하면 같은 동네에 모인다"는 기하학적 성질입니다.

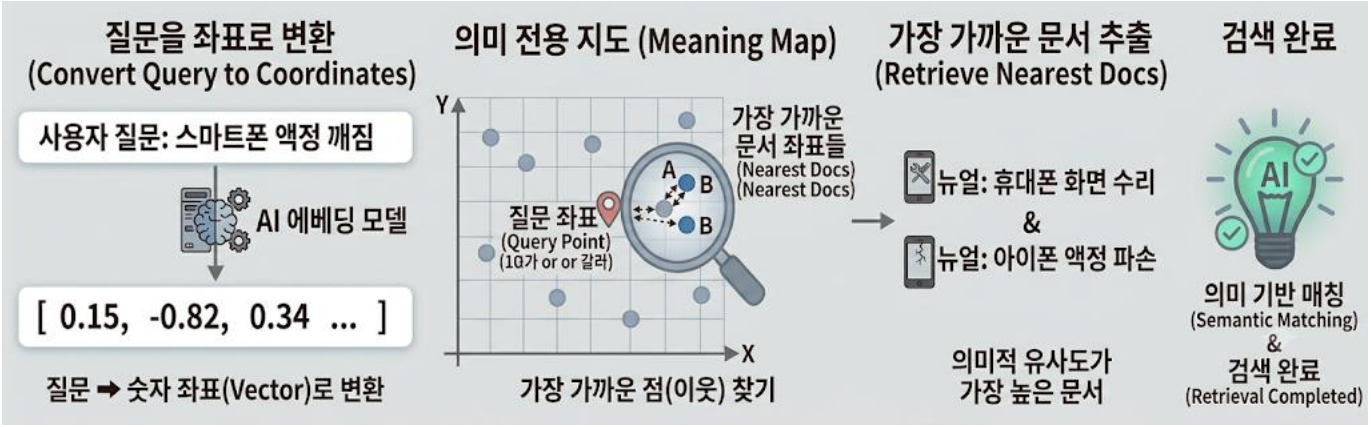
- 실제 위치 비유
  - 문장 A: "스마트폰 액정이 깨졌어요"
  - 문장 B: "휴대폰 화면 파손 수리"
  - 문장 C: "오늘 점심에 맛있는 파스타를 먹었다"
- AI가 좌표 지도에 배치하는 모습
  - 문장 A와 문장 B는 겹치는 글자가 거의 없지만, AI는 두 문장 모두 '휴대용 IT 기기의 부품 손상'이라는 문맥을 가리키고 있음을 알고 있습니다.
  - 따라서 지도 상에서 문장 A와 문장 B는 아주 가까운 좌표에 나란히 점을 찍습니다.
  - 반면 음식 이야기를 하는 문장 C는 저 멀리 좌표에 점을 찍어버립니다.



Vector DB와의 연결점

◉ 이제 컴퓨터는 글자를 읽고 이해할 필요가 없습니다.

- 텍스트를 좌표로 변환: 들어오는 모든 질문과 문서를 AI를 통해 숫자 좌표(Dense Vector)로 바꿉니다.
- 가장 가까운 점 찾기: 사용자가 질문을 던지면, 그 질문의 좌표 주변을 돋보기로 둘러보며 "가장 가까운 위치에 모여 있는 문서 좌표들"을 파악 하면 됩니다.





1. 비정형 데이터와 텍스트 수치화 기법

- 최신 임베딩 모델 생태계 둘러보기

SK 주식회사 AX

임베딩 모델을 고를 때 알아야 할 핵심 기준

◉ 실무에서 모델을 고를 때 눈여겨봐야 할 기준은 딱 세 가지입니다.

- 출력 차원 수 (Dimensions): 문장 하나를 몇 개의 소수점 숫자로 압축하는가? (숫자가 많을수록 섬세하게 표현하지만 Vector DB 메모리를 많이 차지함)
- 최대 입력 길이 (Context Length): 한 번에 몇 글자(토큰)까지 읽고 좌표를 찍을 수 있는가? (단락 수준인가, 긴 문서 전체인가?)
- 클라우드 API vs 자체 구축(오픈소스): 돈을 내고 API로 편하게 쓸 것인가, 사내 서버나 폐쇄망에 직접 올려서 무료로 돌릴 것인가?

OpenAI 최신 임베딩 모델

◉ 대표 라인업:

- 현재 상용 서비스와 프로토타입 개발에서 가장 널리 쓰이는 표준 모델입니다.
- OpenAI의 3세대 임베딩 시리즈는 뛰어난 가성비와 유연성으로 이전 세대(ada-002)를 완전히 대체했습니다.
- text-embedding-3-small (가성비 최우선)
  - 기본 1536개의 숫자로 좌표를 만듭니다.
  - 가격이 매우 저렴하면서도 일반적인 챗봇이나 사내 매뉴얼 검색에 충분한 성능을 냅니다.
- text-embedding-3-large (정밀도 최우선)
  - 기본 3072개의 숫자로 좌표를 만듭니다.
  - 복잡한 기술 문서, 법률/금융 도메인 등 뉘앙스가 중요한 고난도 검색에 적합합니다.

OpenAI 최신 임베딩 모델

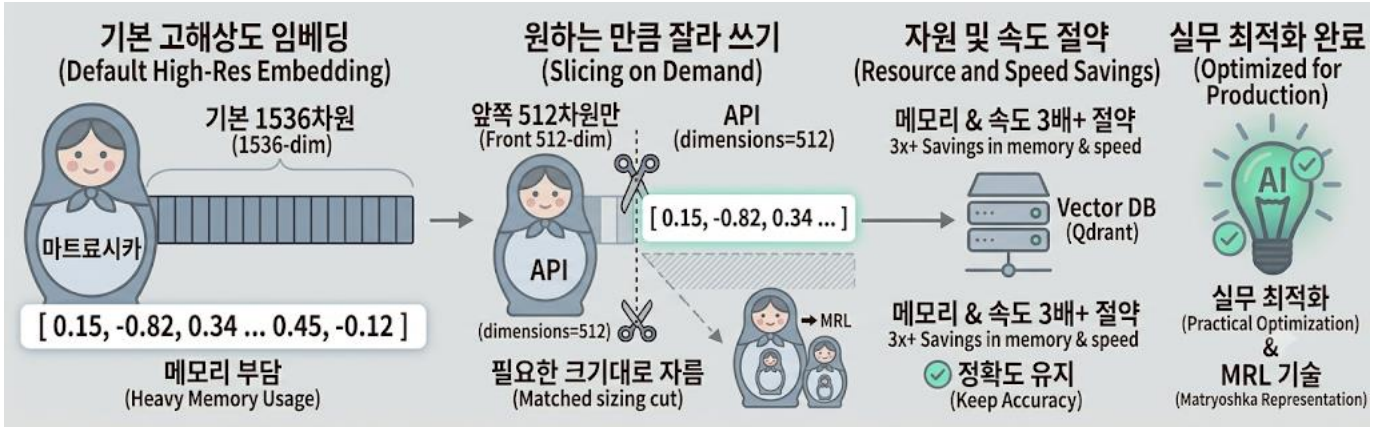
◉ 최대 입력 길이

- 두 모델 모두 한 번에 책 10여 페이지에 달하는 약 8,000토큰(약 2~3만 자) 분량의 장문을 끊김 없이 통째로 이해할 수 있습니다.

OpenAI 최신 임베딩 모델

◉ 원하는 만큼 숫자를 잘라 쓰는 마트료시카(MRL) 기술

- 보통 1536개의 숫자를 DB에 넣으면 메모리가 부담될 수 있습니다.
- OpenAI 3세대 모델은 API를 부를 때 dimensions=512처럼 지정하면, 러시아 인형 '마트료시카'처럼 중요한 정보만 남기고 앞쪽 512개의 숫자만 똑 떼어내서 전달해 줍니다.
- 정확도는 거의 떨어지지 않으면서 Vector DB(Qdrant) 메모리 사용량과 검색 속도를 3배 이상 절약할 수 있는 실무 최적화 기능입니다.



BGE (BAAI General Embedding)

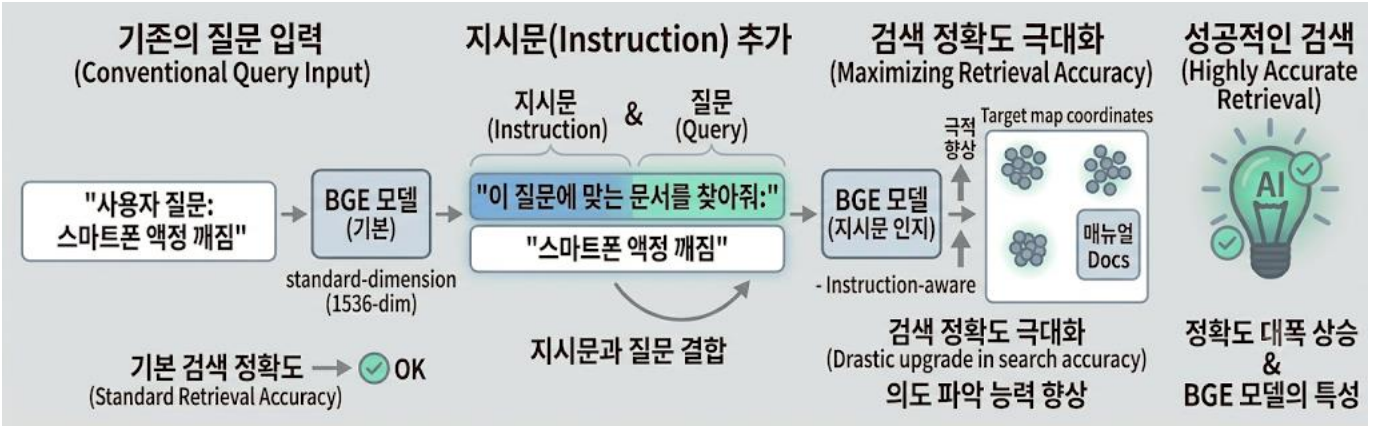
◉ 대표 라인업

- 사내 데이터 유출을 막기 위해 폐쇄망을 쓰거나, OpenAI API 비용이 부담스러운 엔터프라이즈 환경에서 사실상의 표준(De-facto Standard)으로 쓰이는 최강의 오픈소스 모델군입니다.
- BGE v1.5 시리즈 (bge-small, bge-base, bge-large): 모델 크기에 따라 384~1024차원의 가벼운 벡터를 만듭니다. 일반적인 단락 검색에 최적화되어 있습니다.
- BGE-M3
  - Multi-Lingual: 한국어와 영어를 포함해 100개 이상의 언어를 매끄럽게 지원합니다.
  - Multi-Functionality: 단 하나의 모델을 돌렸을 뿐인데 "의미 검색용 좌표(Dense Vector)"와 "정확한 키워드 매칭용 점수(Sparse Vector)"를 동시에 뽑아줍니다.
  - 긴 문서 지원: 최대 8192 토큰을 지원해 긴 글도 넉넉하게 소화합니다.

BGE (BAAI General Embedding)

◉ 실무 특징

- BGE 모델은 질문(Query)을 던질 때 앞에 "이 질문에 맞는 문서를 찾아줘" 같은 지시문(Instruction)을 붙여주면 검색 정확도가 극적으로 올라가도록 훈련되어 있습니다.



E5 시리즈

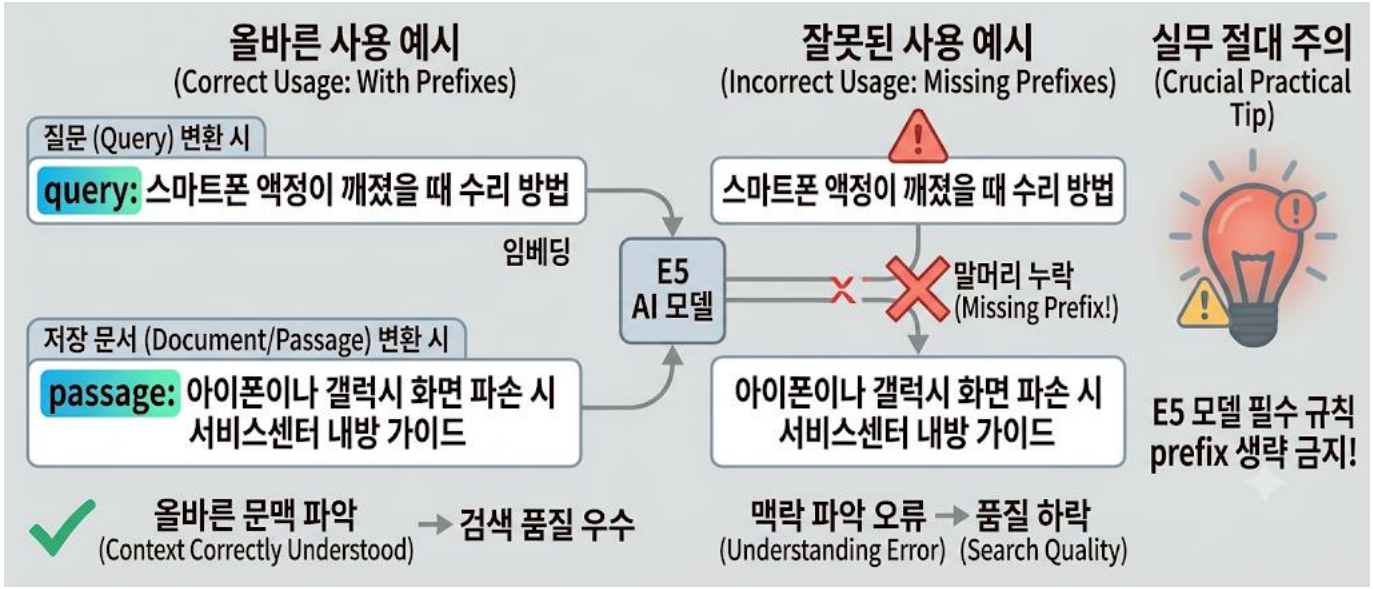
◉ 대표 라인업

- 마이크로소프트(MS)가 웹 상의 수억 개 문서 쌍을 학습시켜 공개한 모델로, 검색과 질문-답변(QA) 시스템에서 압도적인 안정성을 보여줍니다.
- Multilingual-E5 (다국어 모델): 한국어 문맥 이해도가 매우 뛰어나 국내 온프레미스(사내 구축) AI 프로젝트에 자주 도입됩니다.
- E5-Mistral-7b (대형 LLM 기반): 작은 언어 모델이 아니라 70억 개 파라미터를 가진 거대 언어 모델(Mistral)을 임베딩 전용으로 개조한 모델입니다. 성능은 벤치마크 최상위권이지만, 서버에 비싼 고성능 GPU가 필요하다는 것이 특징입니다.

E5 시리즈

◉ 실무 사용 시 절대 주의점

- E5는 모델을 만들 때 엄격한 약속을 걸어두었습니다.
- 질문을 좌표로 바꿀 때는 문장 앞에 반드시 query: 를 붙여야 하고, 저장할 문서는 앞에 passage: 를 붙여서 넘겨야 합니다.
- 이 말머리를 빠뜨리면 모델이 문장의 맥락을 엉뚱하게 파악하여 검색 품질이 뚝 떨어지므로 실무 개발 시 꼭 챙겨야 하는 포인트입니다.



한눈에 보는 실무 선택 기준

◉ "개발 일정이 촉박하고, 복잡한 인프라 관리 없이 빠르게 만들고 싶다":

- 고민 없이 OpenAI text-embedding-3-small 선택 (API 키 하나로 즉시 연동).

◉ "사내 보안 규정상 외부 클라우드로 텍스트를 보낼 수 없는 폐쇄망이다":

- BGE-M3 또는 Multilingual-E5를 사내 GPU 서버(또는 로컬 컨테이너)에 올려서 사용.

◉ "키워드 검색(품번, 고유명사)과 의미 검색을 한 번에 다 잡고 싶다":

- 단일 모델로 두 가지 벡터를 모두 뽑아주는 BGE-M3가 최선의 선택.



2. 벡터 유사도와 Vector DB의 등장

SK 주식회사 AX

2. 벡터 유사도와 Vector DB의 등장

- 유사도 개념

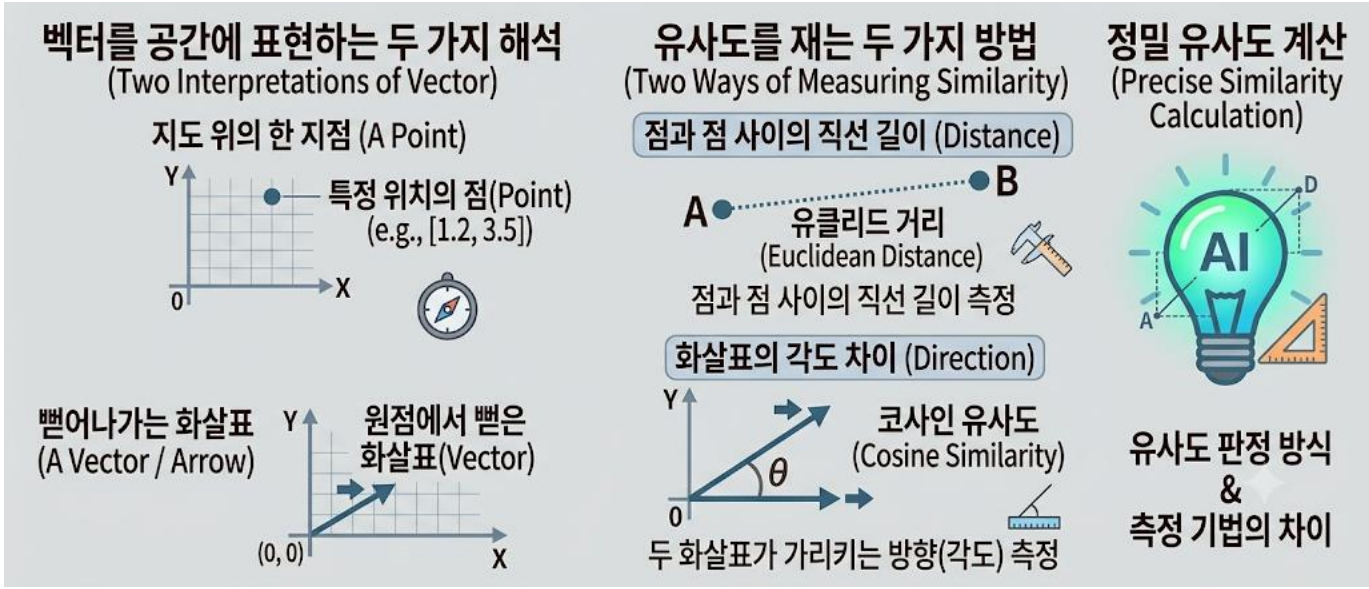
SK 주식회사 AX

벡터를 바라보는 두 가지 시선

◉ 컴퓨터가 글자를 수백 개의 소수점 숫자로 바꾼 것(벡터)을 공간 위에 올려놓으면 두 가지 방법으로 해석할 수 있습니다.

- 지도 위의 한 지점(점): 서울시청, 강남역처럼 특정 위치에 콕 찍힌 점
- 원점(0, 0)에서 뻗어나가는 화살표: 기준점에서 특정 위치를 향해 쭉 뻗은 나침반 바늘

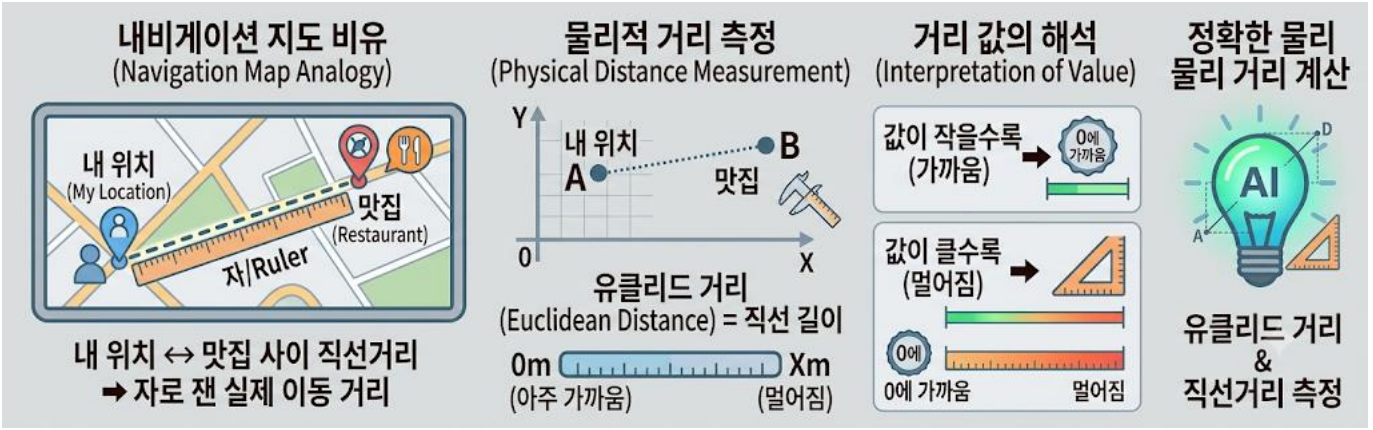
◉ 두 점이 얼마나 가까운지(유사한지)를 잴 때, "점과 점 사이의 직선 길이를 잴 것인가(유클리드 거리)", 아니면 "화살표가 가리키는 방향의 각도를 잴 것인가(코사인 유사도)"에 따라 완전히 다른 결과가 나옵니다.



유클리드 거리 (Euclidean Distance)

◉ 자(Ruler)로 잰 실제 이동 거리

- 내비게이션 지도 위에서 내 위치와 맛집 사이의 직선거리를 자로 잰 것과 같습니다.
- 두 점 사이의 물리적인 간격을 그대로 측정하므로, 값이 작을수록(0에 가까울수록) 서로 가깝고, 숫자가 커질수록 멀리 떨어진 데이터입니다.



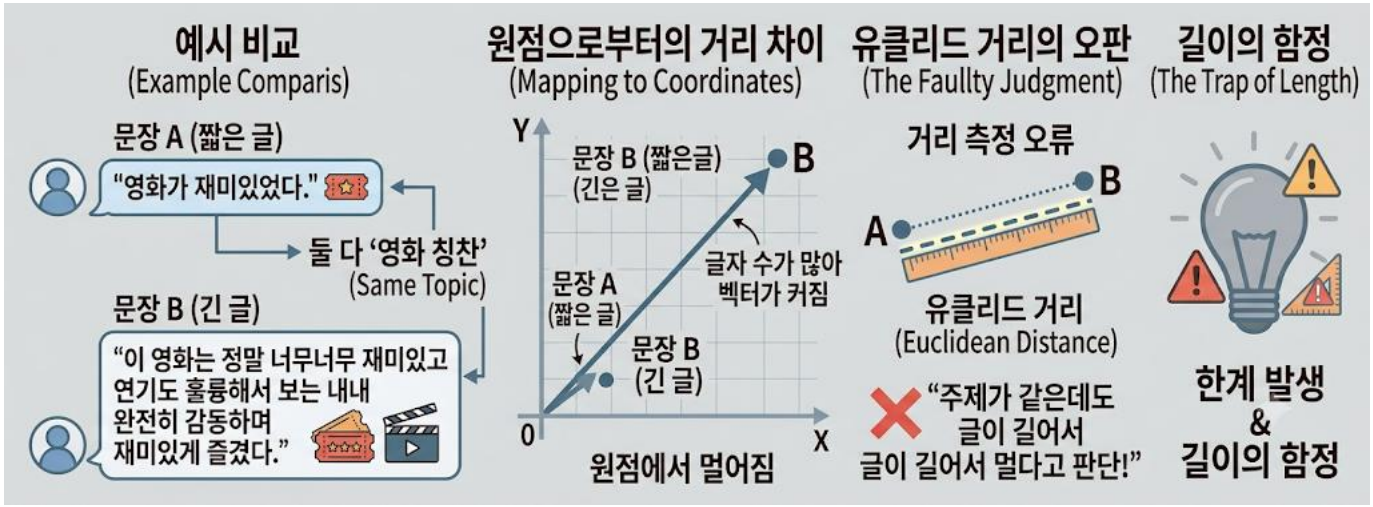
유클리드 거리 (Euclidean Distance)

◉ 자연어(글자) 비교에서 마주치는 함정: "길이의 함정"

- 유클리드 거리는 '내용의 방향'뿐만 아니라 '문장의 길이'에도 직접적인 영향을 받습니다.
- 예시 비교:

  - 문장 A (짧은 글): "영화가 재미있었다."
  - 문장 B (긴 글): "이 영화는 정말 너무너무 재미있고 연기도 훌륭해서 보는 내내 완전히 감동하며 재미있게 즐겼다."

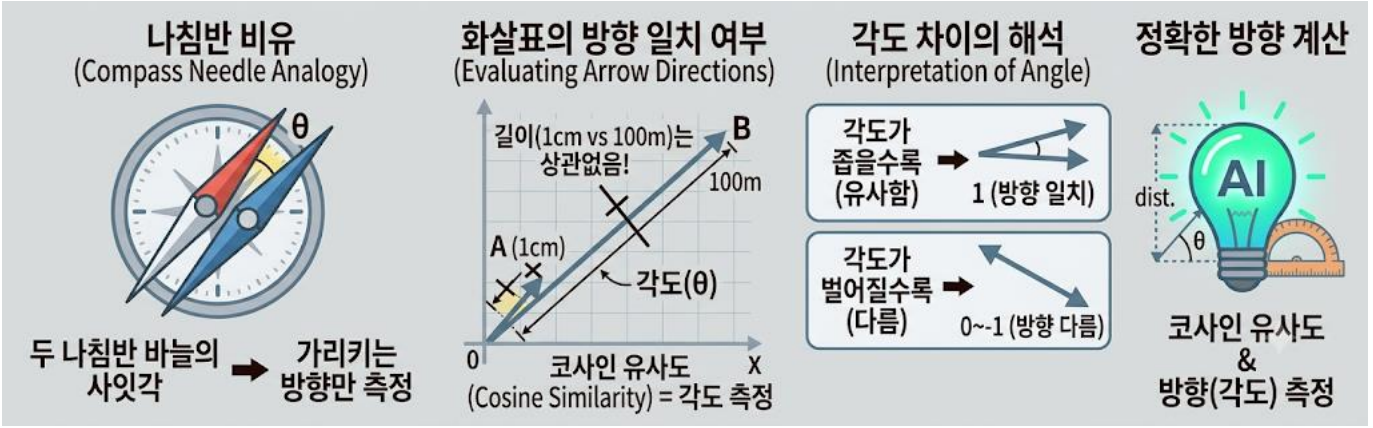
- 문장 A와 문장 B는 똑같이 "영화 칭찬"을 하고 있습니다. 하지만 문장 B는 단어가 훨씬 많아서 벡터의 숫자들이 커지다 보니, 지도 상에서 원점으로부터 훨씬 멀리 떨어진 위치에 점이 찍히게 됩니다.
- 그 결과, 유클리드 거리로 자를 대고 재면 주제가 같은데도 단순히 글자 수가 길다는 이유만으로 두 문장의 거리가 매우 멀다고 오판하게 됩니다.



코사인 유사도 (Cosine Similarity)

◉ 직관적인 비유

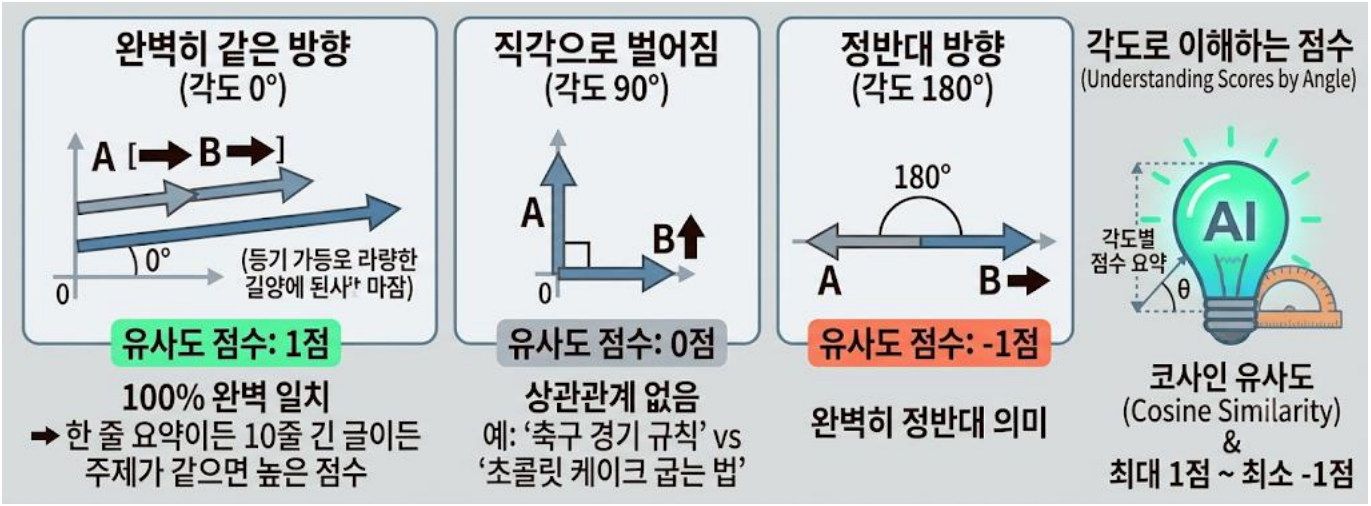
- 두 나침반 바늘의 사잇각
- 벡터를 원점에서 출발하는 화살표로 봅니다.
- 화살표의 길이가 1cm이든 100m이든 전혀 신경 쓰지 않고, "두 화살표가 가리키는 방향(각도)이 얼마나 일치하는가?"만 따집니다.



코사인 유사도 (Cosine Similarity)

◉ 각도로 이해하는 점수 (최대 1점에서 최소 -1점까지)

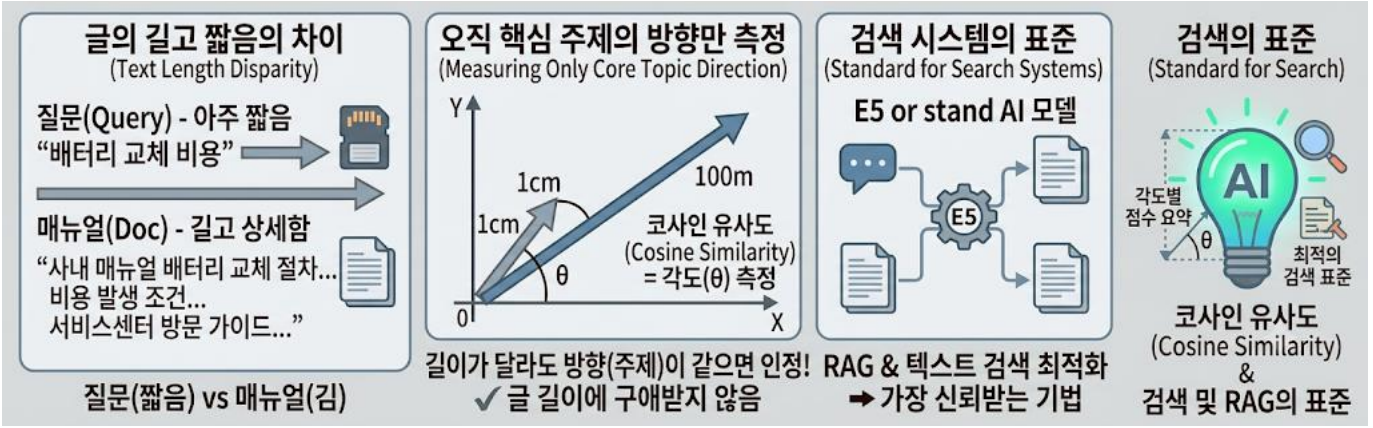
- 두 화살표가 완벽히 같은 방향을 가리킬 때 (각도 0°)
  - 유사도 점수 1점 (100% 완벽 일치)
- 두 화살표가 서로 직각으로 벌어질 때 (각도 90°)
  - 유사도 점수 0점 (서로 아무런 상관관계가 없음)
- 두 화살표가 정반대 방향을 가리킬 때 (각도 180°)
  - 유사도 점수 -1점 (완벽히 정반대 의미)



코사인 유사도 (Cosine Similarity)

◉ 텍스트 검색에서 코사인 유사도가 표준으로 쓰이는 이유

- 검색창에 입력하는 질문은 대개 "배터리 교체 비용"처럼 짧습니다.
- 반면 사내 매뉴얼에 적힌 답변 문단은 몇 줄씩 길게 설명되어 있습니다.
- 코사인 유사도는 글의 길고 짧음에 구애받지 않고 오직 '핵심 주제가 같은 방향을 가리키는가?'만 판별하기 때문에, RAG나 텍스트 검색 시스템에서 많이 사용됩니다.



한눈에 비교하는 차이점

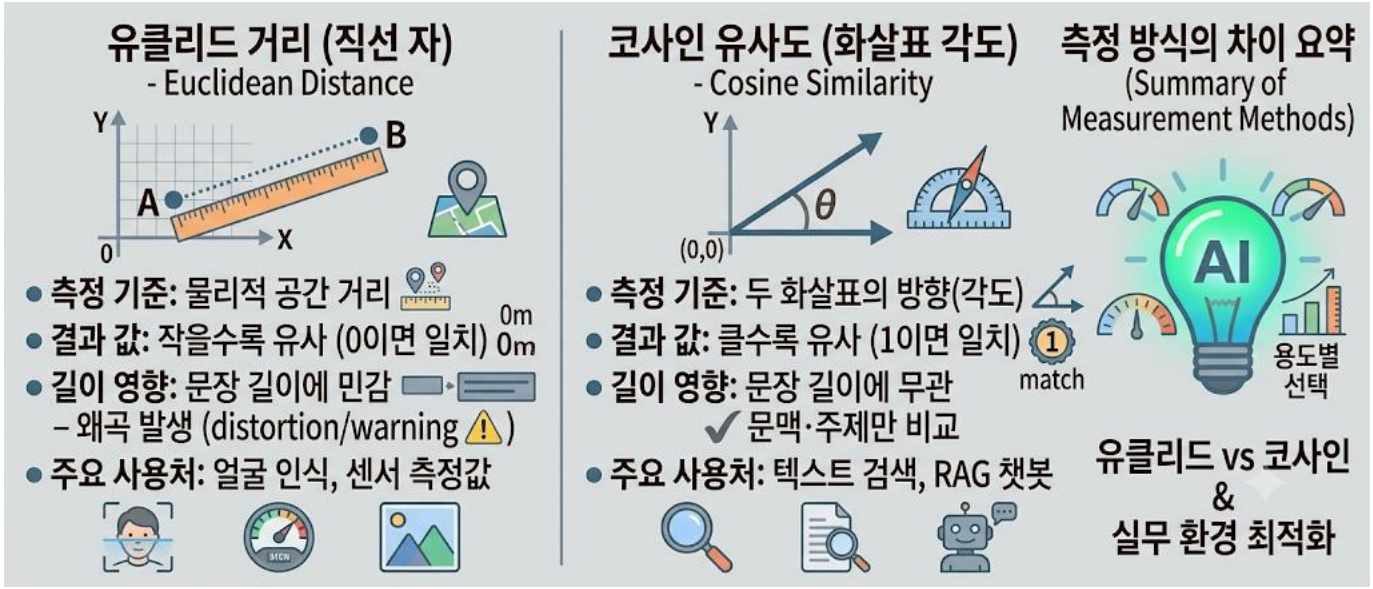
◉ 유클리드 거리 (직선 자)

- 측정 기준: 두 점 사이의 실제 물리적 공간 거리
- 결과 값: 작을수록 유사함 (0이면 같은 위치)
- 길이 영향: 문장 길이에 매우 민감함 (긴 글과 짧은 글 비교 시 왜곡 발생)
- 주요 사용처: 얼굴 인식, 센서 측정값 비교, 이미지 특징점 비교 등 크기 자체가 중요한 분야

◉ 코사인 유사도 (화살표 각도)

- 측정 기준: 두 화살표가 가리키는 방향(각도)
- 결과 값: 클수록 유사함 (1이면 완벽히 같은 방향)
- 길이 영향: 문장 길이에 영향을 받지 않음 (순수한 주제와 문맥만 비교)
- 주요 사용처: 텍스트 검색, 문서 Q&A, RAG 챗봇 시스템

한눈에 비교하는 차이점


유클리드 거리 (직선 자)
- Euclidean Distance
Y
B
A
0
X
● 측정 기준: 물리적 공간 거리
● 결과 값: 작을수록 유사 (0이면 일치)
0m
0m
● 길이 영향: 문장 길이에 민감
– 왜곡 발생 (distortion/warning)
● 주요 사용처: 얼굴 인식, 센서 측정값
코사인 유사도 (화살표 각도)
- Cosine Similarity
Y
θ
(0,0)
● 측정 기준: 두 화살표의 방향(각도)
● 결과 값: 클수록 유사 (1이면 일치)
1
match
● 길이 영향: 문장 길이에 무관
✔ 문맥·주제만 비교
● 주요 사용처: 텍스트 검색, RAG 챗봇
측정 방식의 차이 요약
(Summary of Measurement Methods)
AI
용도별 선택
유클리드 vs 코사인
&
실무 환경 최적화

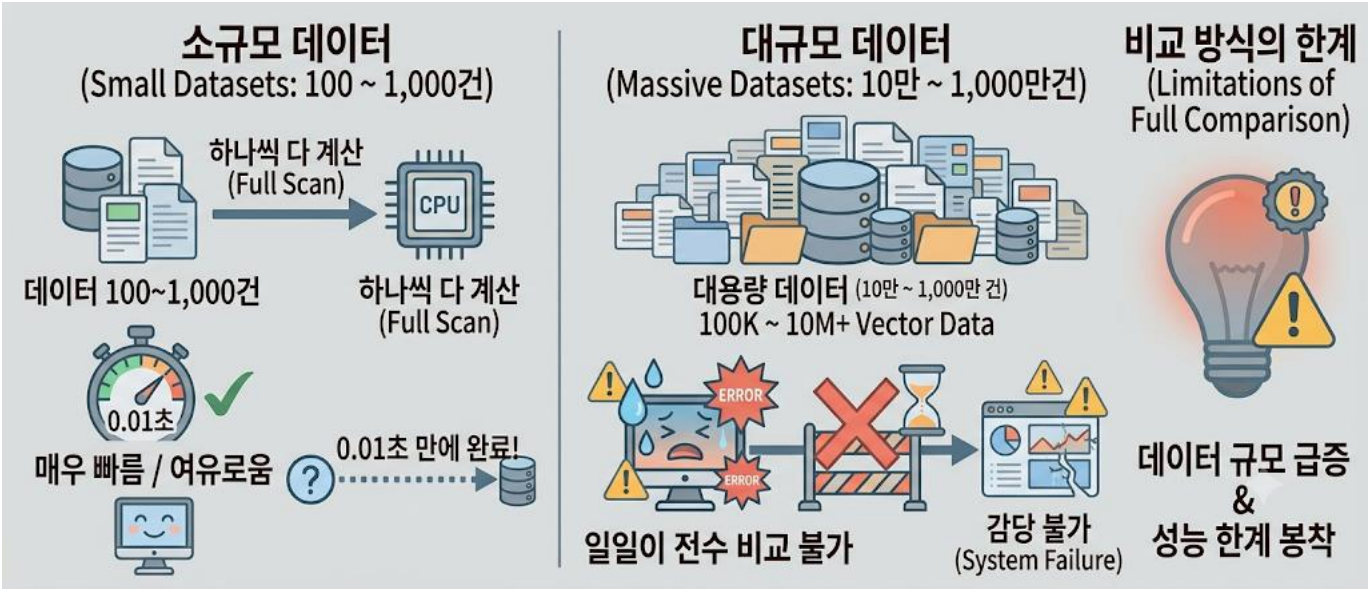
2. 벡터 유사도와 Vector DB의 등장

- 데이터가 수만~수백만 건으로 늘어날 때 생기는 문제

SK 주식회사 AX

전수 조사

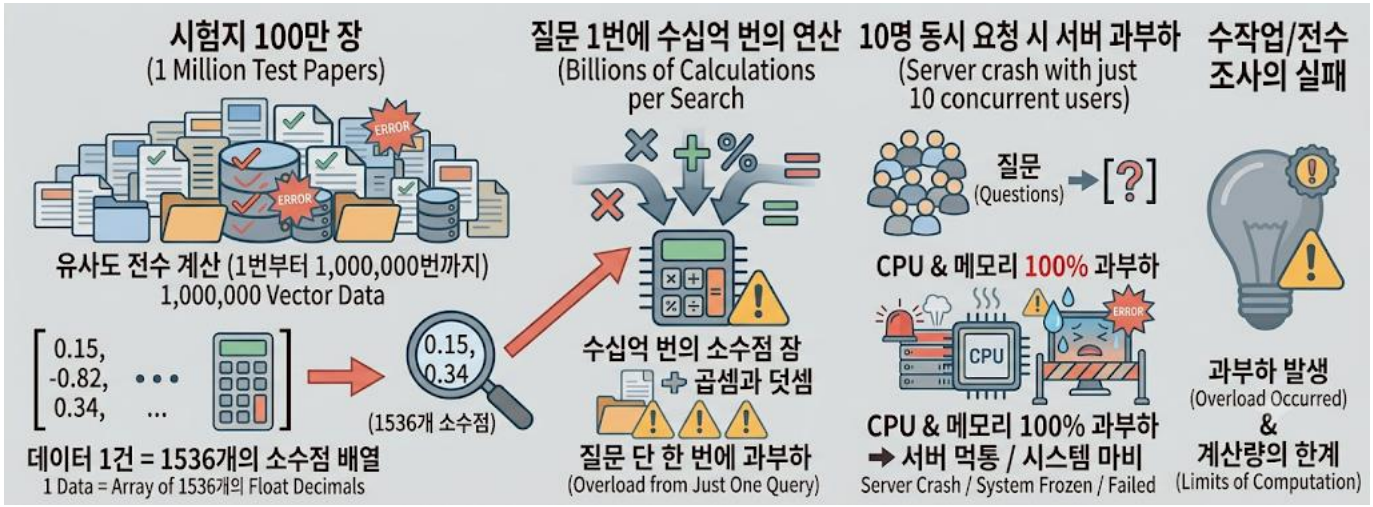
◉ 데이터가 100건, 1,000건 수준일 때는 질문이 들어왔을 때 저장된 모든 벡터와 각도를 하나씩 다 재어봐도 컴퓨터가 0.01초 만에 답을 냅니다. 하지만 실무 환경처럼 데이터가 10만 건, 100만 건, 1,000만 건으로 늘어나면 시스템이 감당하지 못하고 멈춰 서게 됩니다.



전수 조사

◉ 시험지 100만 장을 일일이 채점하는 상황

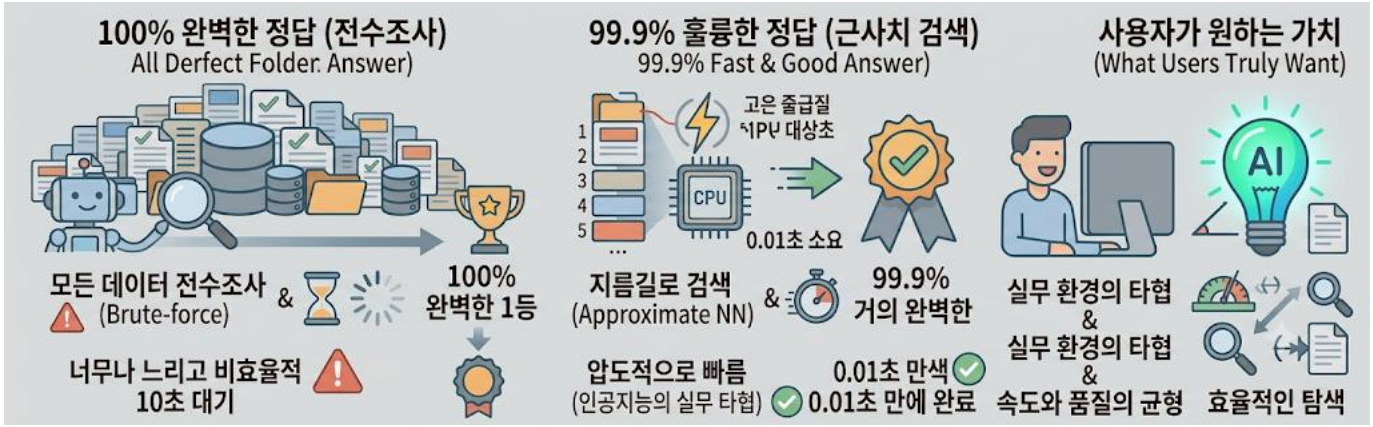
- 질문 하나를 검색할 때마다 100만 개의 데이터와 내 질문 사이의 유사도를 1번부터 1,000,000번까지 단 하나도 빼놓지 않고 전부 계산해야 합니다.
- 게다가 데이터 1건은 숫자 1개가 아니라, 1536개(OpenAI 기준)의 소수점으로 꽉 찬 긴 배열입니다.
- 계산량으로 따지면 질문 단 한 번에 수십억 번의 소수점 곱셈과 덧셈이 일어나며, 사용자가 10명만 동시에 질문을 던져도 CPU와 메모리가 100%를 찍으며 서버가 먹통이 됩니다.



전수 조사

◉ 100% 완벽한 정답의 비효율

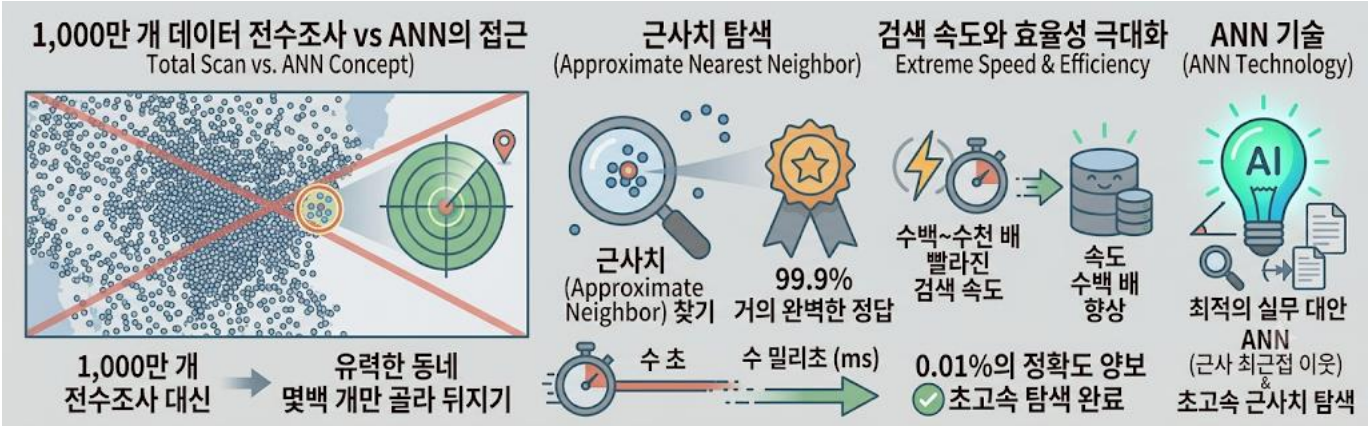
- 모든 데이터를 다 뒤지는 방식(Brute-force)은 "우주에서 가장 완벽하게 일치하는 1등"을 100% 찾아줍니다.
- 하지만 사용자는 검색창을 누르고 10초를 기다려 나오는 '100% 완벽한 정답'보다, 0.01초 만에 눈앞에 뜨는 '99.9% 훌륭한 정답'을 원합니다.



고속 검색(ANN)

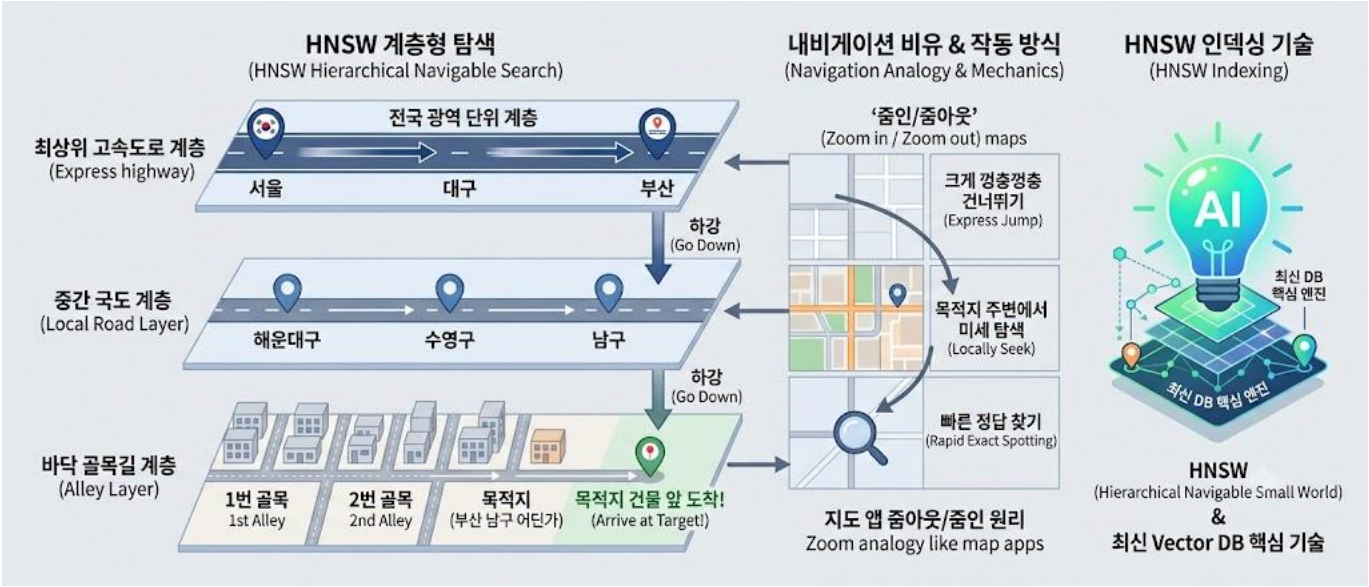
◉ ANN (Approximate Nearest Neighbor, 근사 최근접 이웃)

- 말 그대로 가장 가까운 이웃을 찾되, '근사치(대략 맞닿아 있는 가장 유사한 값)'를 찾는 기술입니다.
- 1,000만 개의 데이터를 다 뒤지는 미련한 방식을 버리고, "정답이 있을 법한 유력한 지역만 콕 집어서 몇백 개만 골라 뒤져보자"는 아이디어입니다.
- 0.01%의 극단적인 정확도를 양보하는 대가로, 검색 속도를 수백 배에서 수천 배(수 초→ 수 밀리초) 빠르게 끌어올립니다.



HNSW 알고리즘의 동작 원리

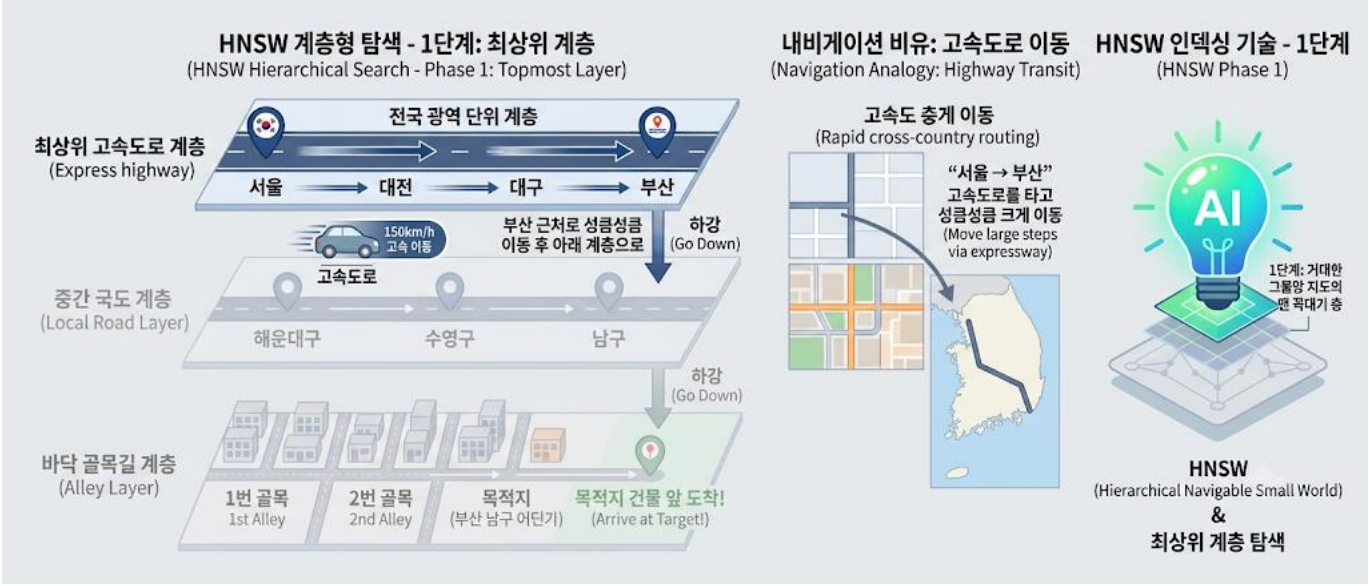
◉ 현재 Qdrant를 비롯한 최신 Vector DB들이 가장 핵심 엔진으로 채택하고 있는 ANN 인덱싱 기술이 바로 HNSW (Hierarchical Navigable Small World)입니다.



HNSW 알고리즘의 동작 원리

◉ 1단계: 최상위 계층

- HNSW는 데이터를 저장할 때 여러 층(Layer)의 거대한 그물망(그래프) 지도를 만듭니다.
- 맨 꼭대기 층에는 아주 굵직굵직한 대표 도시(서울, 대전, 대구, 부산)들만 듬성듬성 연결해 둡니다.
- 서울에서 출발해 부산에 있는 특정 건물을 찾아갈 때, 처음부터 시골 골목길을 하나하나 다 뒤지지 않고 일단 고속도로를 타고 150km/h로 부산 근처까지 성큼성큼 크게 이동합니다.



HNSW 알고리즘의 동작 원리

◉ 2단계: 중간 계층 (시/군/구 지도 - 시내 도로 진입)

- 부산 근처에 도착하면 한 층 아래 지도(국도 및 시내 도로망)로 줌인(Zoom-in)해서 내려옵니다.
- 이제 해운대구인지, 수영구인지 구 단위로 좁혀가며 빠르게 방향을 잡습니다.



HNSW 알고리즘의 동작 원리

◉ 3단계: 최하위 계층 (골목길 지도 - 목적지 돋보기 탐색)

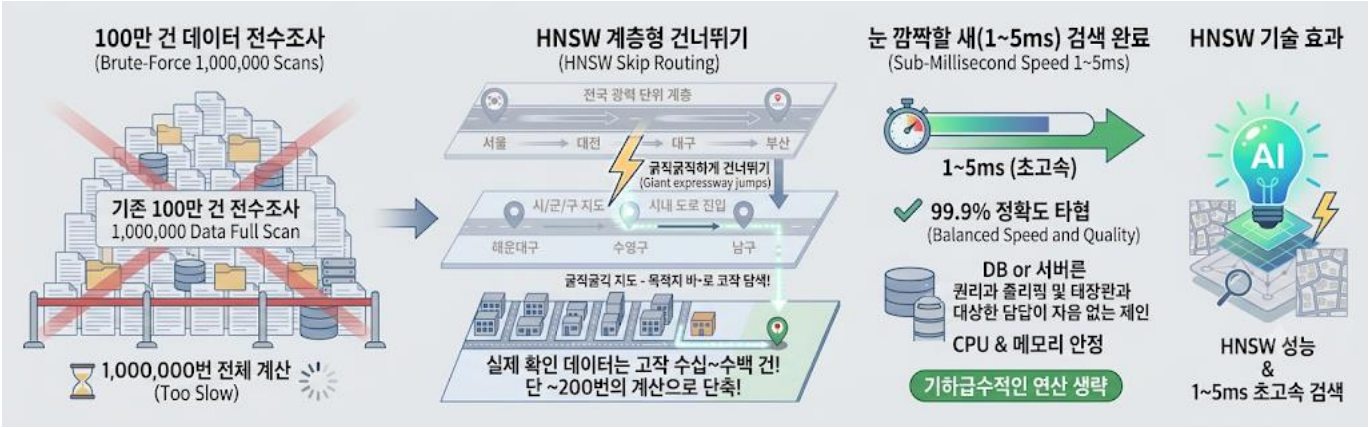
- 목적지 동네에 도달하면 가장 바닥 층인 골목길 지도로 내려옵니다.
- 이 동네에는 수많은 골목길(밀집된 벡터들)이 촘촘히 거미줄처럼 이어져 있습니다.
- 이미 목적지 바로 코앞까지 와 있기 때문에, 주변에 있는 집 몇십 군데만 확인하고 가장 가까운 정답 건물을 최종 확정합니다.



HNSW가 실무에서 좋은 이유

◉ 기하급수적인 연산 생략:

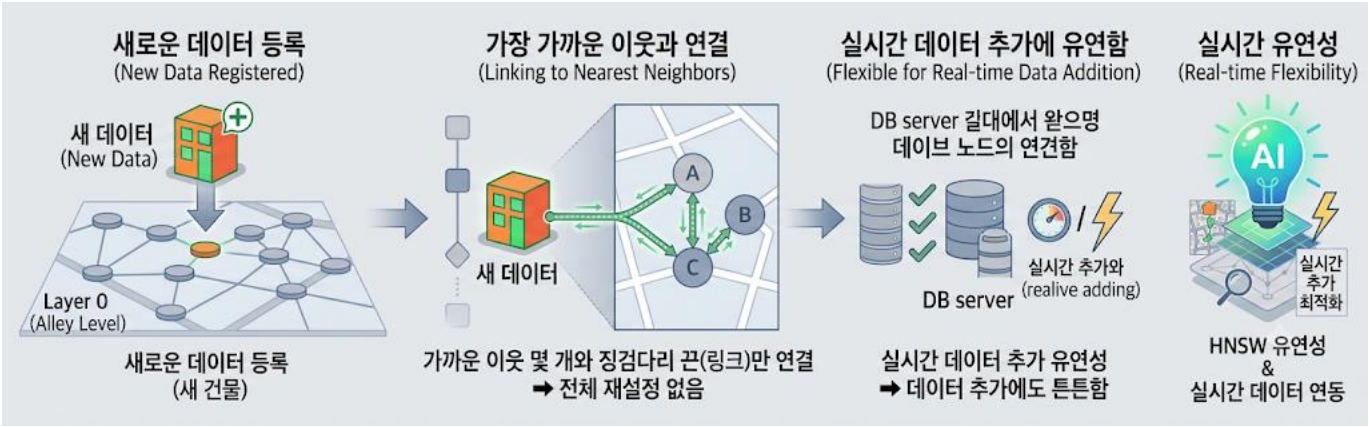
- 100만 건의 데이터가 있어도, 고속도로에서 굵직하게 건너뛰며 이동하기 때문에 실제 계산해보는 데이터는 고작 수십~수백 건에 불과합니다.
- 100만 번 계산할 것을 200번 만에 끝내버리므로 검색이 눈 깜짝할 새(1~5ms)에 끝납니다.



HNSW가 실무에서 좋은 이유

◉ 데이터가 추가되어도 튼튼함

- 새로운 데이터(새 건물)가 등록되면, 가장 가까운 이웃 몇 개와 징검다리 끈(링크)만 연결해 주면 되므로 실시간 데이터 추가에도 유연하게 대처합니다.



2. 벡터 유사도와 Vector DB의 등장

- 일반 RDBMS와 Vector DB의 차이점

SK 주식회사 AX

일반 RDBMS와 Vector DB의 차이점

◉ 일반 RDBMS (MySQL, PostgreSQL 등)

- 글자나 숫자가 똑같은지 비교하는 DB
- 동작 방식: 데이터의 정확한 일치(Exact Match)나 명확한 범위 비교($=, <, >, \text{LIKE}$)를 기반으로 동작합니다.
- 검색 예시
  - WHERE user_id = 100 (100번 사용자를 정확히 찾기)
  - WHERE price <= 50000 (5만 원 이하 데이터만 추려내기)
  - WHERE title LIKE '%스마트폰%' ('스마트폰'이라는 글자가 똑같이 적혀 있는 행 찾기)
- 한계: 사용자가 "아이폰 화면 깨짐"을 검색했을 때 문서에 "스마트폰 액정 파손"이라고 적혀 있다면, 글자 모양이 일치하지 않으므로 RDBMS는 관련 문서를 찾아내지 못합니다.

일반 RDBMS와 Vector DB의 차이점

◉ Vector DB (Qdrant)

- 의미가 비슷한지 공간 좌표를 찾는 DB
- 동작 방식: 텍스트를 AI를 통해 수백~수천 개의 소수점 숫자로 된 '의미 좌표(벡터)'로 바꾼 뒤, 다차원 공간 상에서 "질문 좌표와 가장 가까운 이웃 데이터"를 기하학적으로 탐색(ANN)합니다.
- 검색 예시
  - "주말에 가볍게 웃으며 볼 만한 영상 추천해줘"→ 글자가 겹치지 않아도 코미디 장르 영화 줄거리가 높은 유사도 순으로 추출됨
- 핵심 가치: 사람이 질문을 대충 던지거나 오타가 섞여 있어도, 질문의 '의도와 뉘앙스'를 파악해 가장 그럴듯한 맥락의 데이터를 낚아채 옵니다.

수많은 Vector DB 중 'Qdrant'를 쓰는 이유

◉ 가볍고 빠른 성능 (Rust 언어 기반 고성능 엔진)

- 언어적 강점: Qdrant는 시스템 프로그래밍 언어인 Rust(러스트)로 밑바닥부터 작성되었습니다.
- 낮은 메모리 오버헤드: Java/Go 기반의 다른 엔진들과 달리 가비지 컬렉터(GC)로 인한 멈춤 현상(Stop-the-world)이 없으며, CPU 연산과 메모리를 극도로 효율적으로 사용합니다.
- 손쉬운 배포: 무거운 종속성 없이 단일 바이너리(또는 가벼운 Docker 이미지 하나)만으로 구동되어, 로컬 노트북 환경부터 대규모 클라우드 클러스터까지 동일하게 가볍고 안정적으로 동작합니다.

수많은 Vector DB 중 'Qdrant'를 쓰는 이유

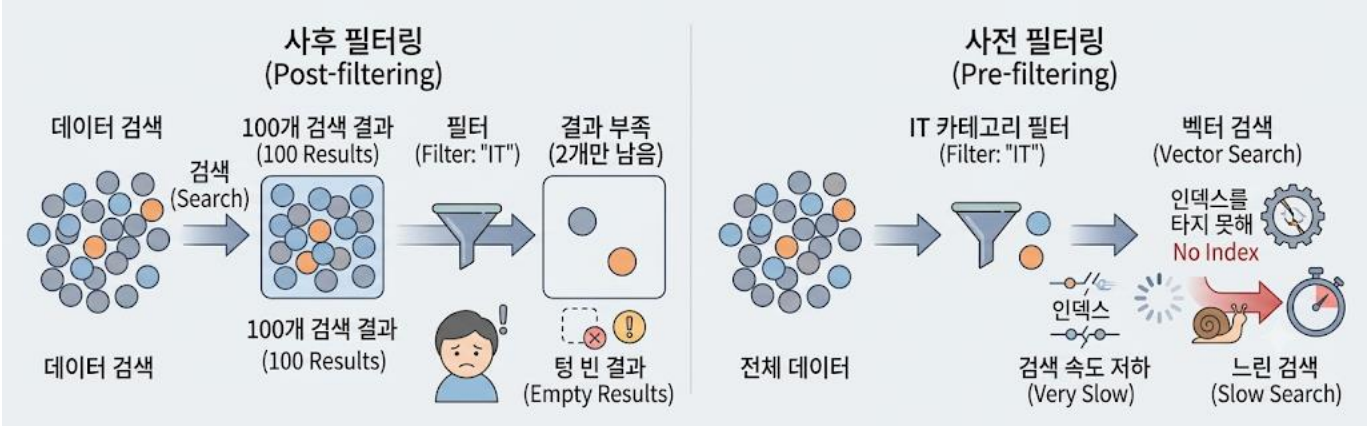
◉ 직관적인 웹 대시보드

- 내장 Web UI 기본 제공: 별도의 복잡한 시각화 툴이나 추가 컨테이너를 설치할 필요 없이, Qdrant를 띄우고 브라우저로 http://localhost:6333/dashboard에 접속하면 즉시 웹 화면이 열립니다.
- 직접 눈으로 보는 검증:
  - 내가 만든 컬렉션(Collection)과 적재된 포인트(Point)의 개수를 실시간 확인
  - 저장된 텍스트와 메타데이터(Payload)가 어떻게 들어가 있는지 JSON 형태로 열람
  - 대시보드 내의 콘솔 창에서 REST API 검색 쿼리를 직접 날려보고 결과를 시각적으로 테스트 가능

수많은 Vector DB 중 'Qdrant'를 쓰는 이유

◉ 기존 방식들의 문제점

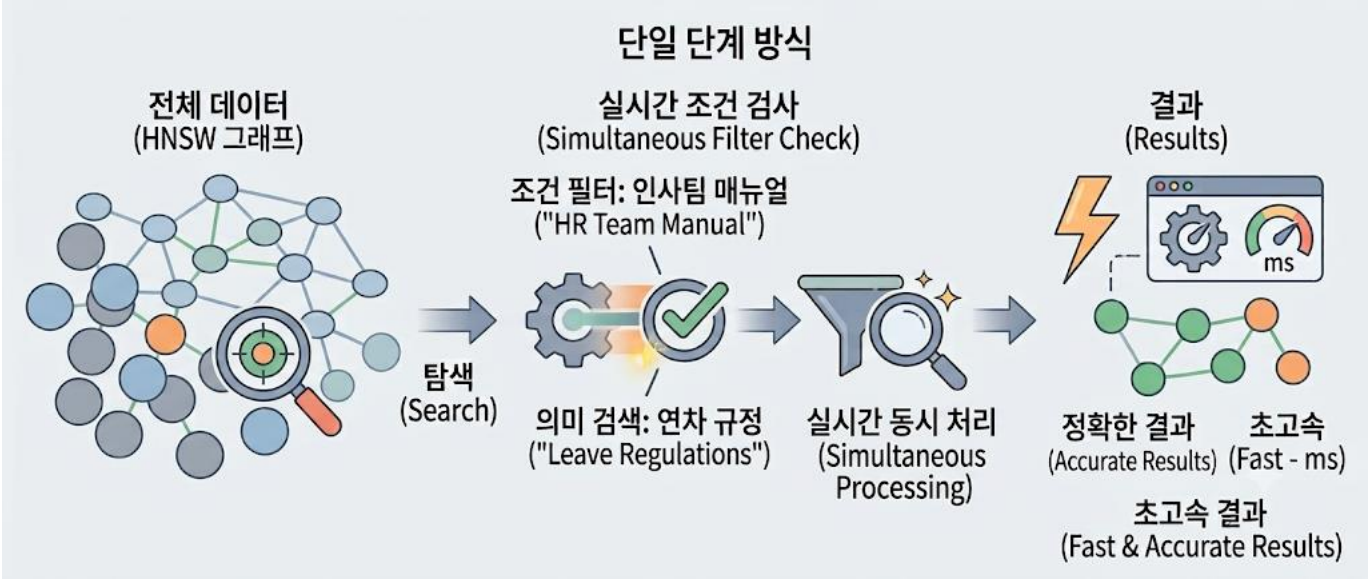
- 사후 필터링 (Post-filtering): 의미가 가까운 100개를 먼저 뽑은 뒤 "카테고리가 IT인 것"을 걸렀더니 남는 데이터가 2개밖에 없어서 결과가 텅 비는 문제
- 사전 필터링 (Pre-filtering): "카테고리가 IT인 것"을 일반 DB처럼 먼저 다 걸러낸 뒤 벡터 검색을 돌렸더니 인덱스를 타지 못해 검색 속도가 급격히 느려지는 문제



수많은 Vector DB 중 'Qdrant'를 쓰는 이유

◉ Qdrant의 Single-stage 방식:

- HNSW 그래프를 타고 가까운 벡터를 찾아 내려가는 탐색 과정 도중에 실시간으로 메타데이터 조건을 함께 검사합니다.
- 따라서 "인사팀 매뉴얼 중에서(조건 필터) + 연차 규정과 가장 의미가 비슷한 문단(의미 검색)"을 정확도 손실 없이 밀리초 단위로 초고속 처리합니다.



3. RAG 아키텍처와 Qdrant 핵심 기능

SK 주식회사 AX



3. RAG 아키텍처와 Qdrant 핵심 기능

- LLM의 거짓말(환각)을 막는 RAG(검색 증강 생성)

SK 주식회사 AX

LLM은 왜 거짓말(환각, Hallucination)을 할까?

◉ ChatGPT 같은 거대 언어 모델(LLM)은 백과사전이나 데이터베이스가 아닙니다. 모델의 본질은 "앞에 나온 단어들 뒤에 가장 그럴듯하게 이어질 다음 단어를 확률적으로 예측하는 기계"입니다.

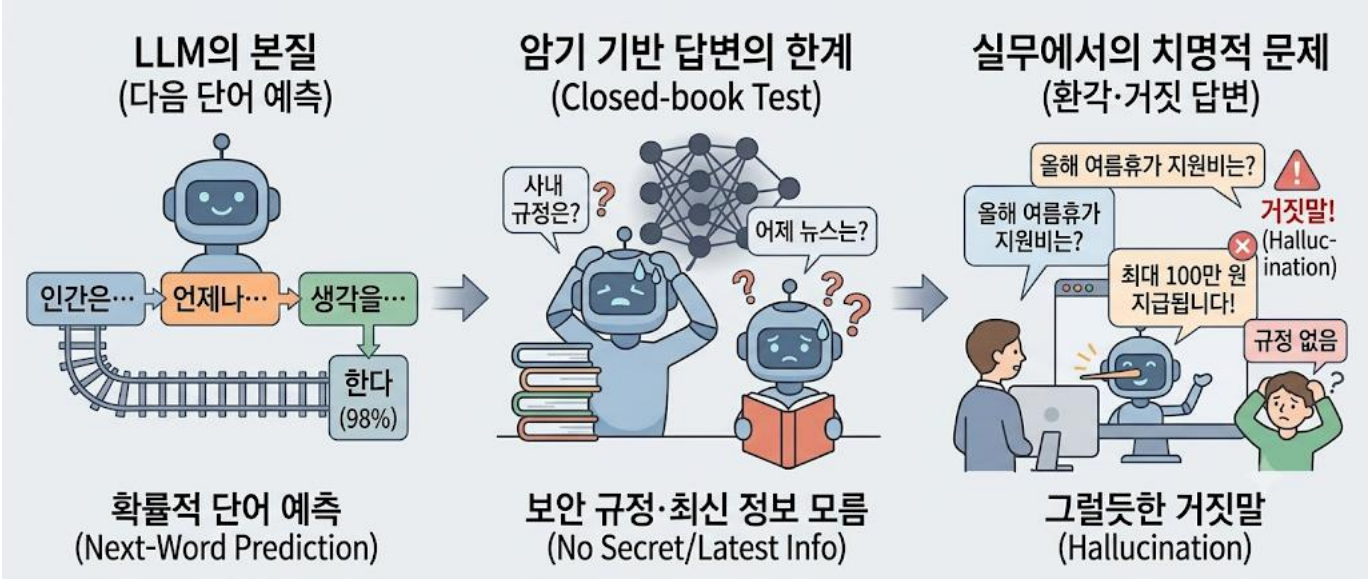
◉ 암기 기반 시험(Close-book Test)의 한계

- LLM은 학습할 때 수많은 책과 인터넷 문서를 읽었지만, 그 방대한 지식을 머릿속(신경망의 가중치)에 압축해서 외워둔 상태입니다.
- 자신이 잘 모르는 질문(사내 보안 규정, 어제 발생한 최신 뉴스, 본 적 없는 개인정보)을 받더라도, 모른다고 답하기보다 확률적으로 가장 매끄럽고 그럴듯해 보이는 문장을 즉석에서 지어내어 답변합니다.

LLM은 왜 거짓말(환각, Hallucination)을 할까?

◉ 실무에서의 치명적 문제

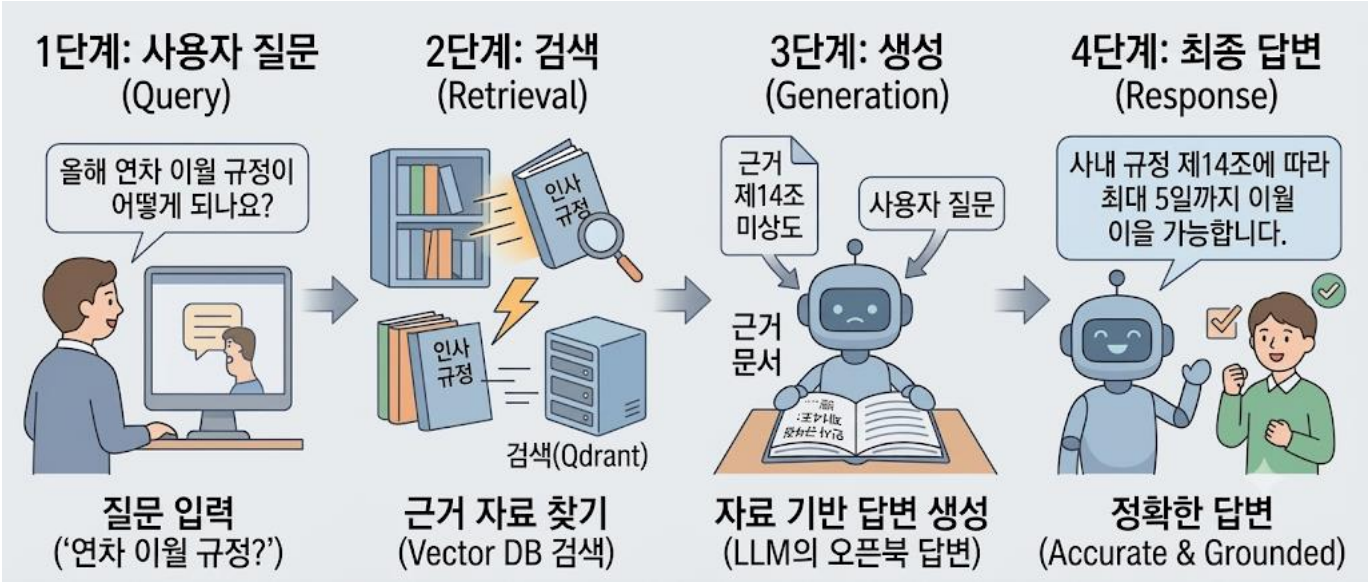
- 직원이 "우리 회사 올해 여름휴가 지원비 규정이 어떻게 돼?"라고 물었을 때, LLM이 사내 규정을 전혀 모른 채 "네, 최대 100만 원까지 영수증 청구 시 지급됩니다"처럼 너무나 당당하고 완벽한 문장으로 거짓말(환각)을 지어내면 심각한 업무 혼선이 발생합니다.



RAG(검색 증강 생성)의 핵심 원리

◉ 이 문제를 해결하기 위해 등장한 기술이 바로 RAG (Retrieval-Augmented Generation, 검색 증강 생성)입니다.

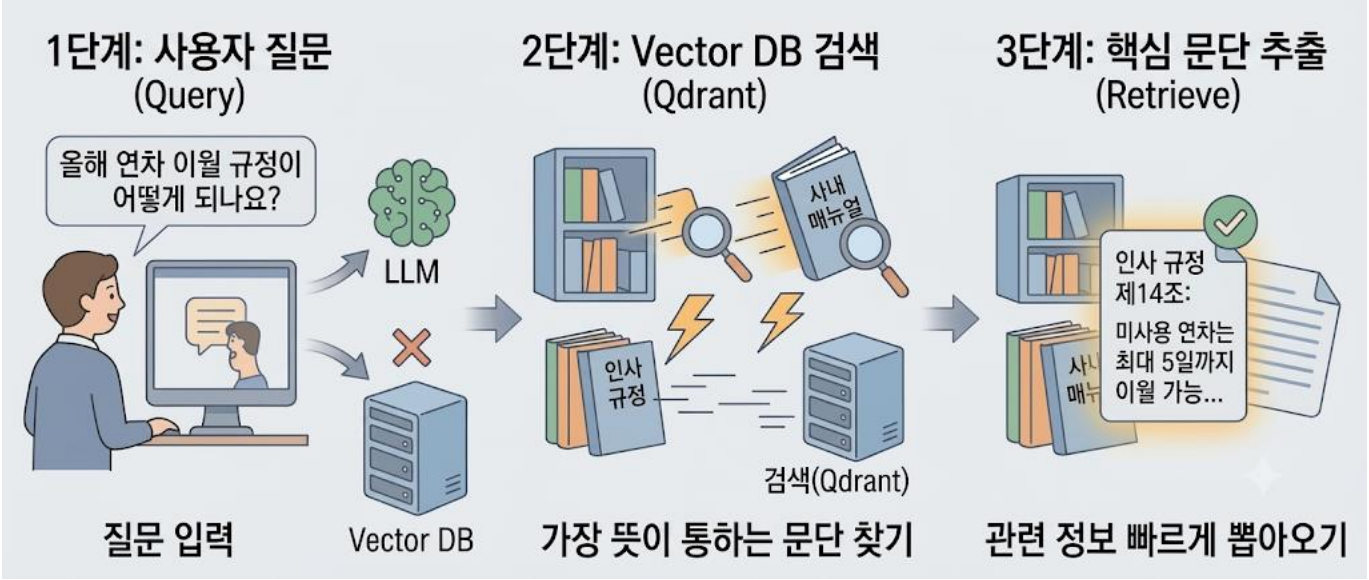
- RAG의 원리는 아주 단순합니다. "외워서 시험 보게 하지 말고, 책(매뉴얼)을 펼쳐놓고 보고 쓰게(오픈북) 하자!"는 것입니다.



RAG(검색 증강 생성)의 핵심 원리

◉ **Retrieval (검색 - 책에서 찾기)**

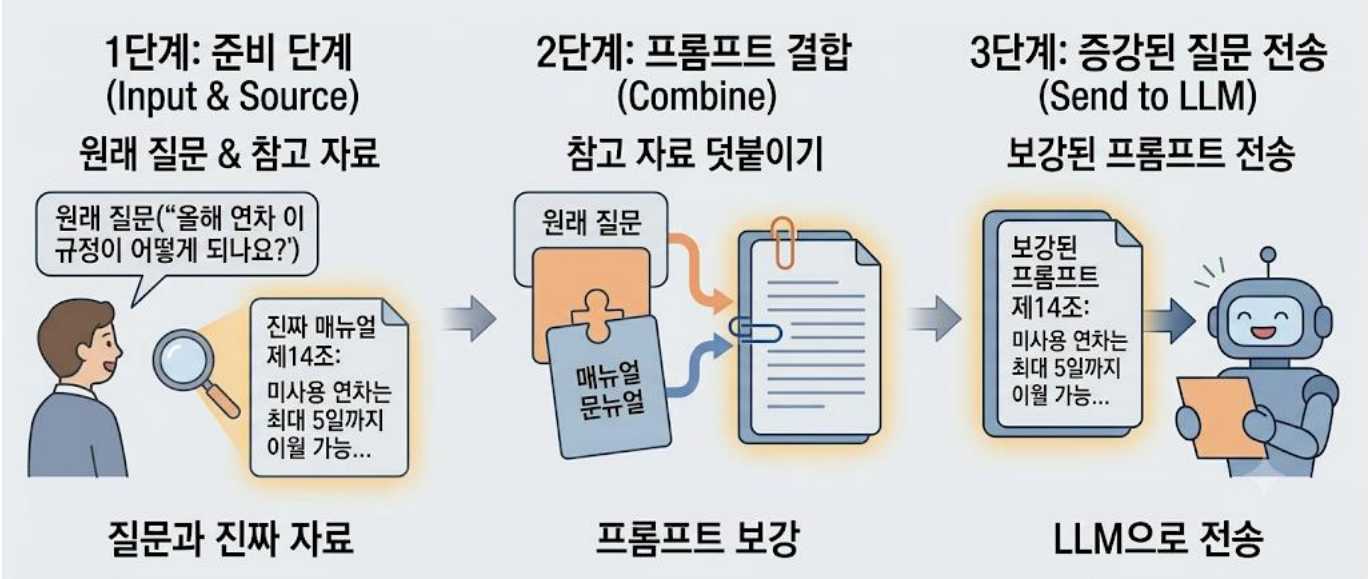
- 사용자가 질문을 던지면, LLM에게 바로 물어보지 않습니다.
- 먼저 회사 매뉴얼들 데이터가 저장되어 있는 Vector DB(Qdrant)에서 질문과 가장 뜻이 통하는 핵심 문단 몇 개를 빠른 속도로 뽑아옵니다.



RAG(검색 증강 생성)의 핵심 원리

◉ **Augmented (증강 - 참고 자료 얹어주기):**

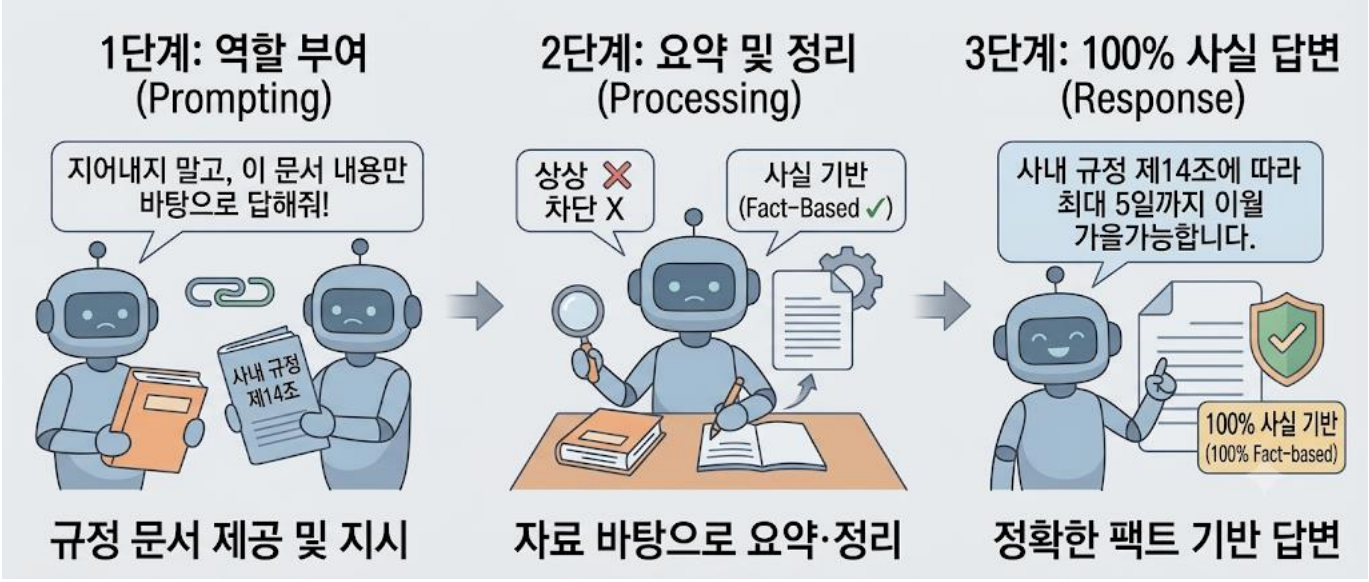
- 사용자의 원래 질문 뒤에, 방금 찾아낸 '진짜 매뉴얼 문단(신뢰할 수 있는 사실)'을 참고 자료로 덧붙여서 프롬프트를 보강합니다.



RAG(검색 증강 생성)의 핵심 원리

◉ Generation (생성 - 보고 쓰기):

- LLM에게 역할을 부여합니다: "너의 상식으로 지어내지 말고, 내가 방금 건네준 이 사내 규정 문단만을 바탕으로 사용자에게 친절하게 요약해서 답해줘."
- LLM은 주관적인 상상력을 발휘할 필요 없이, 주어진 문서를 요약·정리·번역하여 100% 사실에 기반한 정답을 작성합니다.



RAG가 기업 실무에서 절대적인 표준이 된 이유

◉ 환각 현상(거짓말)의 극적인 감소:

- 근거가 되는 출처(Ground Truth)를 눈앞에 두고 글을 쓰기 때문에 엉뚱한 거짓말을 할 위험이 사라집니다.

◉ 비싼 모델 재학습(Fine-tuning)이 필요 없음:

- 사내 규정이 바뀌거나 신제품이 출시되어도 수천만~수억 원을 들여 AI 모델을 다시 학습시킬 필요가 없습니다.
- 그저 새로운 문서를 Qdrant에 등록하거나 기존 문서를 수정/삭제하기만 하면, LLM은 다음 검색부터 즉시 최신 규정을 반영하여 답변합니다.

◉ 출처 제시를 통한 신뢰성 확보:

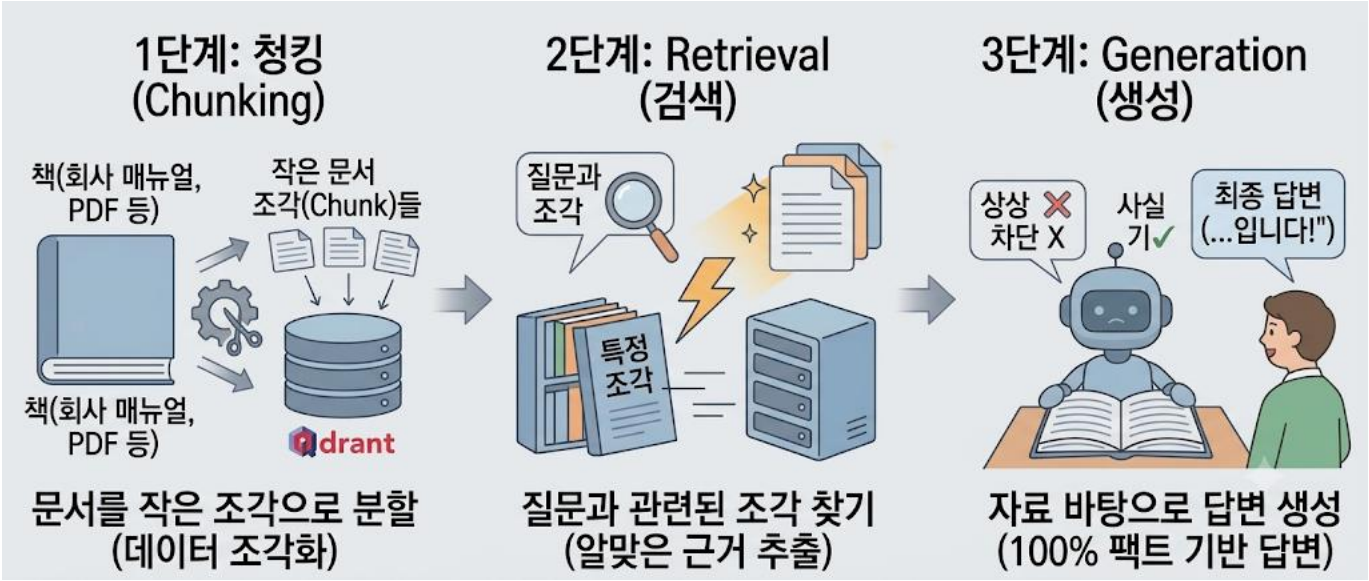
- 답변 끝에 "본 답변은 '2026 인사규정_v2.pdf 14페이지'를 참고하여 작성되었습니다"와 같이 원본 링크와 근거를 명확하게 사용자에게 제시할 수 있습니다.

3. RAG 아키텍처와 Qdrant 핵심 기능

- RAG 파이프라인 3단계

SK 주식회사 AX

RAG 파이프라인 3단계



1단계 청킹(Chunking)

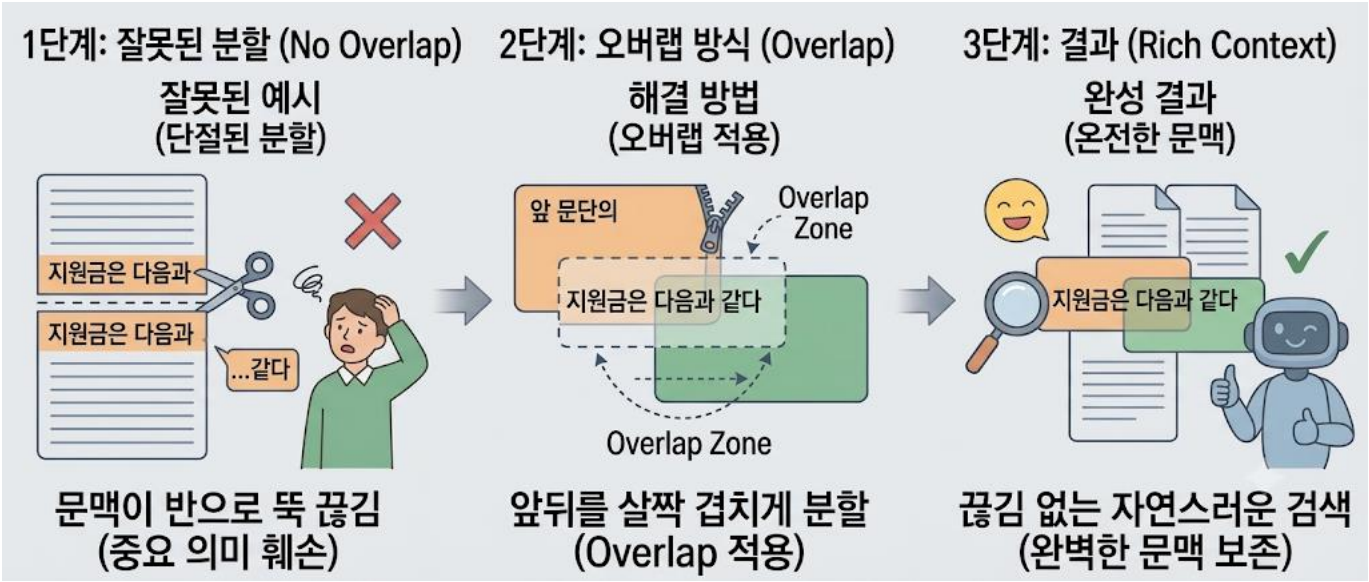
◉ 왜 문서를 통째로 넣지 않고 잘라야 할까?

- 수십~수백 페이지에 달하는 사내 규정집이나 기술 매뉴얼을 통째로 임베딩 모델에 넣으면 두 가지 문제가 발생합니다.
  - 첫째, 모델마다 한 번에 읽을 수 있는 글자 수(입력 토큰 수)에 한계가 있습니다.
  - 둘째, 책 한 권 전체를 하나의 좌표로 압축해 버리면 온갖 주제가 뒤섞여 그 문서가 '연차 규정'을 말하는지, '출장비 정산'을 말하는지 고유한 특징이 흐려집니다.

1단계 청킹(Chunking)

◉ 실무 꿀팁 (오버랩, Overlap)

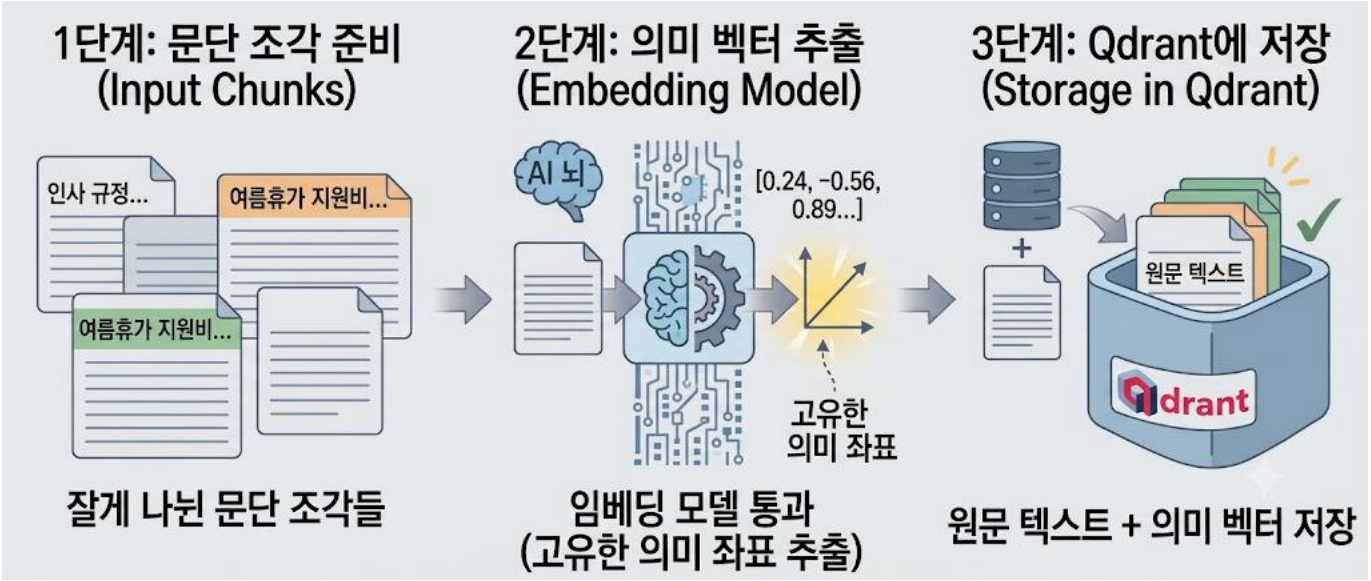
- 가위로 글을 자르다 보면 하필 중요한 문장이 반으로 뚝 끊겨 의미가 훼손될 수 있습니다 (예: 앞 문단 끝에 "지원금은 다음과...", 뒷 문단 처음에 "...같다").
- 그래서 문단을 나눌 때는 앞 문단의 끝부분 1~2문장을 뒷 문단의 시작 부분과 살짝 겹치게(Overlap) 잘라 문맥이 단절되지 않도록 만듭니다.



1단계 청킹(Chunking)

◉ 이 단계의 최종 결과물:

- 잘게 나뉜 각 문단 조각들을 임베딩 모델에 통과시켜 '고유한 의미 좌표(벡터)'를 뽑아낸 뒤, 문단 원문 텍스트와 함께 Qdrant에 저장합니다.



2단계 Vector DB 검색 (Retrieval)

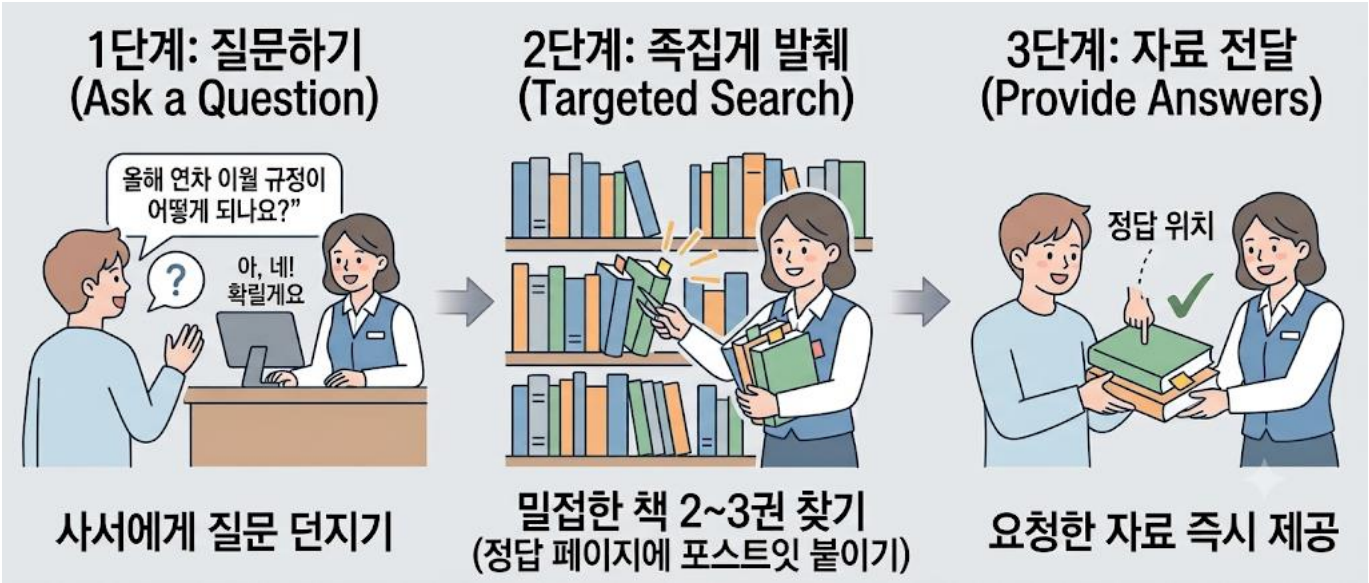
◉ 질문도 똑같이 좌표로 변환

- 사용자가 챗봇에 "입사 첫해인데 여름휴가 며칠 쓸 수 있나요?"라고 질문을 입력합니다.
- 문서를 저장할 때 사용했던 것과 동일한 임베딩 모델에 질문 문장을 넣어 질문의 의미 좌표(벡터)를 생성합니다.

◉ Qdrant의 고속 탐색

- 질문 좌표를 Qdrant에게 건네면, Qdrant 내부의 HNSW 엔진이 수만~수십만 개의 저장된 문단 좌표들 사이를 고속도로 타듯 빠르게 헤집고 다닙니다.
- 질문과 화살표 방향(코사인 유사도)이 가장 일치하는 상위 3~5개의 핵심 문단 조각(Top-K)을 수 밀리초 만에 가져옵니다.

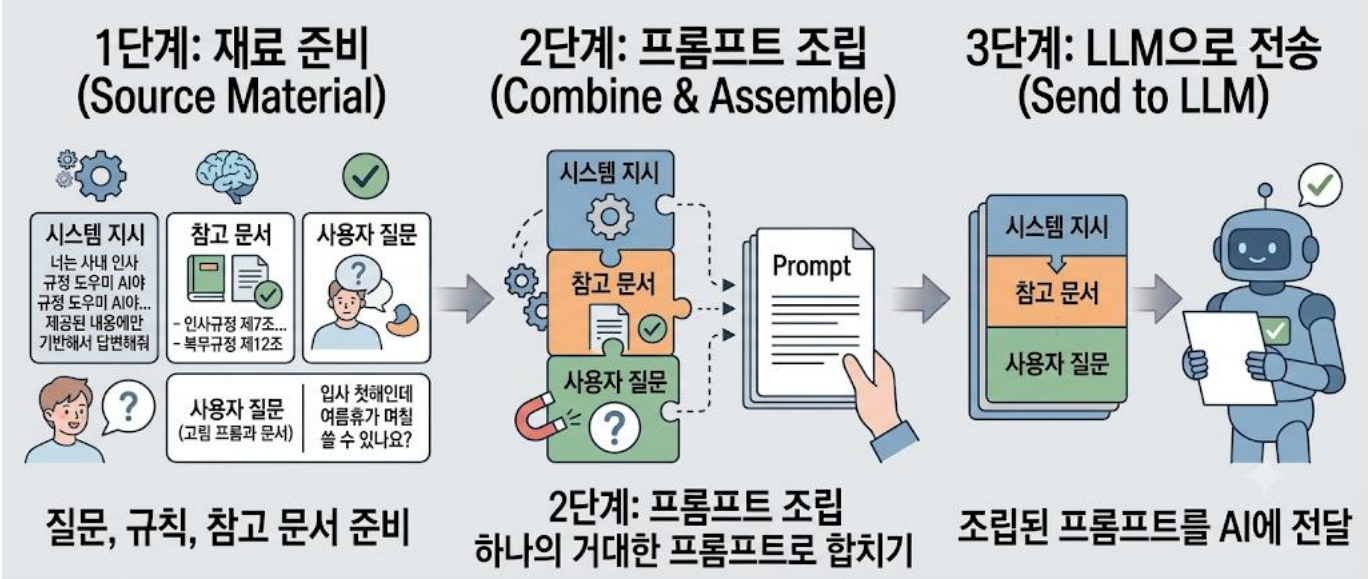
2단계 Vector DB 검색 (Retrieval)



3단계 생성 (Generation)

◉ 프롬프트 조립

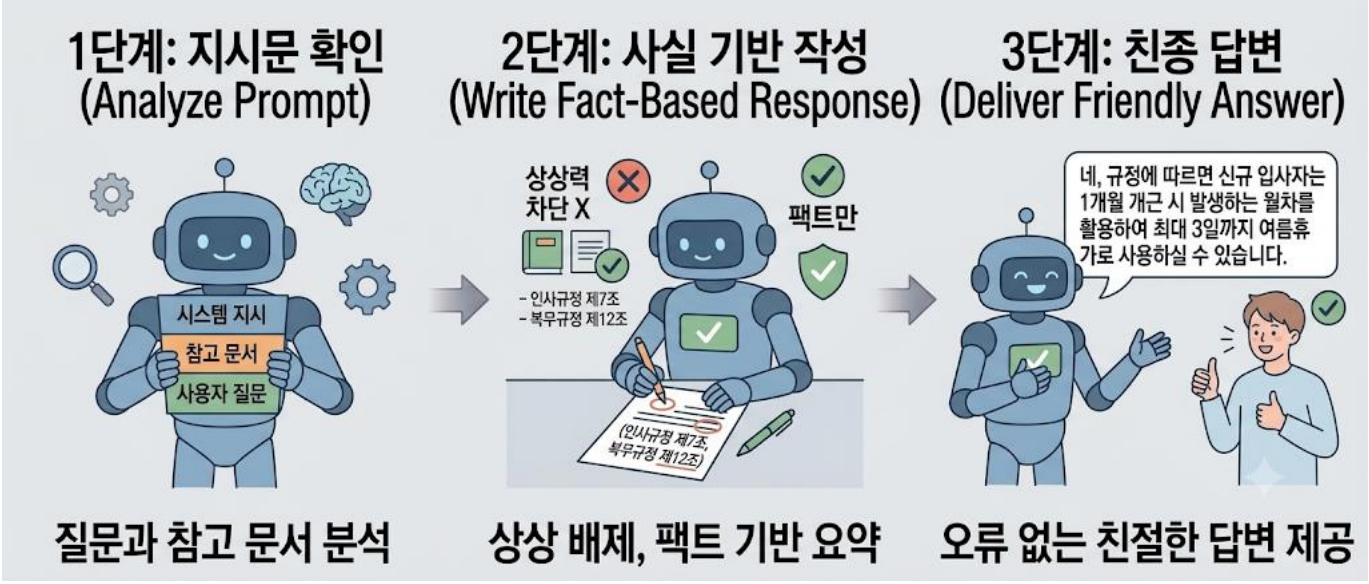
- 2단계에서 Qdrant가 찾아온 실제 매뉴얼 문단들을 사용자의 원래 질문과 하나로 합쳐서 거대 언어 모델(ChatGPT 등)에게 전달할 지시문(Prompt)을 만듭니다.



3단계 생성 (Generation)

◉ LLM의 사실 기반 답변 생성:

- LLM은 자신의 주관이나 상상력을 동원할 필요 없이, 눈앞에 놓인 [참고 문서]의 팩트를 기반으로 수려하고 친절한 문장을 작성합니다.
- "네, 인사규정에 따르면 신규 입사자는 1개월 개근 시 발생하는 월차를 활용하여 최대 3일까지 여름휴가로 사용하실 수 있습니다."



3. RAG 아키텍처와 Qdrant 핵심 기능

- Qdrant의 핵심 구성 요소

SK 주식회사 AX

Qdrant의 데이터 구조

◉ Collection (컬렉션): 엑셀 파일의 '시트'나 데이터베이스의 '테이블'에 해당하는 독립된 데이터 저장 공간입니다.

◉ Point (포인트): 엑셀 표의 가로 '한 줄(Row)'에 해당하는 고유한 데이터 1건입니다.

◉ Payload (페이로드): 데이터 뒤에 붙는 이름표로, 엑셀의 각 '열(Column) 셀'에 채워 넣는 세부 정보입니다.

◉ Vector (벡터): 기존 DB에는 없던 요소로, AI가 글이나 사진의 의미를 수백~수천 개의 소수점 숫자로 압축해 낸 의미 좌표입니다.

Collection (컬렉션)

◉ 개념

- 엑셀 파일 안의 시트나 데이터베이스의 테이블에 해당하는 최상위 저장 단위입니다.
- 사내 시스템을 만든다면 사내 매뉴얼용 컬렉션(company_manuals), 고객 문의용 컬렉션(customer_faq)처럼 용도별로 독립된 방을 만들어 데이터를 격리 보관합니다.

◉ 컬렉션을 생성할 때 고정하는 2가지 규칙:

- 벡터 차원 수: 이 저장소에 들어올 데이터가 소수점 몇 개짜리 좌표인지 지정합니다 (예: OpenAI 3세대 소형 모델을 쓴다면 1536으로 고정).
- 거리 측정 기준: 유사도를 계산할 방식을 결정합니다 (예: 텍스트 검색의 표준인 COSINE 또는 사전 정규화된 고속 내적 DOT).

◉ 주의점: 한 컬렉션 안에는 크기(차원)가 완전히 일치하는 벡터만 들어갈 수 있습니다. 1536차원 보관함에 384차원 오픈소스 모델로 만든 좌표를 함께 섞어 넣을 수는 없습니다.

Point (포인트)

◉ 개념:

- Qdrant 공간에 콕 찍히는 데이터 1건(점)을 의미하며, 엑셀 표의 가로 한 줄 전체를 의미합니다.

◉ Point를 이루는 3대 요소:

- ID (고유 식별자): 다른 데이터와 겹치지 않는 주민번호 같은 고유 번호입니다 (정수 1, 2, 3... 또는 고유 문자열 UUID 사용).
- Vector (좌표): 문단이나 이미지를 AI 임베딩 모델에 넣어 뽑아낸 실수 숫자들의 묶음입니다 (예: [0.014, -0.082, 0.125, ...]).
- Payload (메타데이터): 이 좌표가 원래 어떤 글이었고 어디서 왔는지를 적어둔 부가 정보 묶음(JSON)입니다.

```
{
  "id": 42,
  "vector": [0.0142, -0.0821, 0.1254, 0.0039],
  "payload": {
    "title": "2026 연차 사용 가이드",
    "department": "인사팀",
    "page": 5,
    "text": "미사용 연차는 당해 연도 말일까지 신청 시 최대 5일까지
이월이 가능합니다."
  }
}
```

Payload (페이로드)

◉ 개념:

- 벡터 자체는 컴퓨터 눈에 그저 의미 없는 소수점 숫자 수천 개의 나열일 뿐입니다.
- "이 벡터가 원래 어떤 내용이고, 어느 부서의 문서이며, 언제 작성되었는가?"를 사람이 읽을 수 있는 형태로 벡터 뒤에 붙여둔 메타데이터(이름표)입니다.

◉ Payload가 수행하는 핵심 역할 2가지:

- 원본 텍스트 복원 (RAG 답변 생성의 핵심 재료)
  - Vector DB가 질문과 가장 유사한 좌표를 찾아내더라도 원본 글을 모르면 LLM에게 넘겨줄 수가 없습니다.
  - Payload 안에 원본 문단을 보관해 두면, 검색 성공 즉시 해당 텍스트를 꺼내 LLM 프롬프트에 바로 채워 넣을 수 있습니다.
- 정밀한 조건 필터링:
  - 단순 의미 검색을 넘어, "부서가 '인사팀'이고, 등록 연도가 '2026'인 문서 중에서만 의미가 가장 가까운 데이터를 골라줘"와 같이 강력한 제한 조건을 걸어줍니다.

3. RAG 아키텍처와 Qdrant 핵심 기능

- 실무 필수 기능, 조건 필터링(Payload Filtering)

SK 주식회사 AX

실무에서 단순 벡터 검색만으로는 부족한 이유

◉ 실제 기업 서비스에서 사용자가 던지는 질문은 순수한 의미 검색만으로 끝나지 않습니다. 반드시 업무적인 제한 조건(도메인, 권한, 날짜, 카테고리)이 함께 따라옵니다.

◉ 실무 질문의 실제 형태

- "작년(2025년) 이후에 작성된 규정 중에서만 찾아줘"
- "내가 접근 권한을 가진 '영업본부' 문서 중에서만 골라줘"
- "'기술지원팀' 문서 중에서, '네트워크 연결 오류 해결법'과 가장 의미가 통하는 내용을 찾아줘"

◉ 조건 없이 의미 검색만 돌렸을 때 생기는 대참사

- 질문과 의미가 가장 통하는 상위 3개를 뽑았더니, 1등은 인사팀 문서, 2등은 총무팀 문서, 3등은 마케팅팀 문서가 나오는 상황이 발생합니다.
- 사용자는 분명 "기술지원팀 문서"를 원했는데, 의미상 그럴듯한 엉뚱한 부서의 내용이 섞여 나와 서비스의 신뢰도가 무너집니다.

과거 벡터 검색 엔진들의 실패 방식

◉ 사후 필터링 (Post-Filtering)의 허무한 결과

- 일단 의미가 가장 가까운 벡터 100개를 먼저 뽑아옵니다.
- 그 100개 중에서 "기술지원팀" 이름표를 가진 것만 추려냅니다.
- 문제점: 뽑아온 100개 중에 기술지원팀 문서가 하필 딱 1개뿐이었다면, 사용자는 1개의 결과만 보게 되거나 결과창이 텅 비어버리는 심각한 품질 저하(재현율 급감)가 일어납니다.

◉ 사전 필터링 (Pre-Filtering)의 속도 붕괴

- 일반 DB처럼 "기술지원팀" 문서 10만 건을 먼저 싹 골라냅니다.
- 그 10만 건 안에서 벡터 유사도 계산을 돌립니다.
- 문제점: 미리 만들어둔 초고속 고속도로 지도(HNSW 인덱스)가 깨져버려, 10만 건을 일일이 전수 조사(Brute-force)해야 하므로 검색 속도가 수 초 이상으로 멈추듯 느려집니다.

Qdrant의 해결책

◉ Qdrant가 엔터프라이즈 실무에서 극찬받는 가장 큰 이유가 바로 이 필터링 문제를 가장 영리하게 해결했기 때문입니다.

◉ 고속도로를 달리며 톨게이트 검문소를 동시에 통과하는 원리

- Qdrant는 HNSW 그래프 계층을 타고 "의미상 가장 가까운 목적지"를 향해 고속으로 질주하는 탐색 과정 도중에 실시간으로 이름표(Payload)를 검사합니다.
- 그래프 상에서 다음 이웃 노드로 점프할 때마다 "너 기술지원팀 맞니?"를 동시에 체크하여, 조건에 맞지 않는 길은 애초에 쳐다보지도 않고 조건을 만족하는 노드들만을 징검다리 삼아 목적지까지 번개처럼 도달합니다.

◉ 개발자가 얻는 결과:

- 사용자가 요청한 "기술지원팀 문서(엄격한 조건)"라는 규칙을 100% 만족하면서도,
- HNSW의 초고속 밀리초(ms) 단위 검색 속도를 전혀 희생하지 않고 완벽하게 결과를 반환합니다.



4. 실습

SK 주식회사 AX

실습

◉ 실습환경 준비

- Docker 로 Qdrant 를 실행합니다.
- UV를 통해 파이썬 가상환경을 구축하고 라이브러리를 설치합니다.

◉ 실습1

- Collection을 생성하고 간단한 CRUD를 진행합니다.
- DashBoard를 통해 Collection 내에 저장된 데이터를 확인합니다.

◉ 실습2

- 주어진 문장들을 임베딩하여 저장합니다.
- 문장을 검색하여 봅니다.

실습

◉ 실습3

- 문서를 검색할 때 필터링을 적용하여 봅니다.
- 가벼운 하이브리드 검색을 실습합니다.



5. 종합실습

SK 주식회사 AX

종합실습

◉ 사내 메뉴얼 기반 사마트 Q&A 검색기 만들기

- 문서 쪼개기 : 사내 **FAQ/규정** 텍스트를 문맥 단위로 나누고 부서명, 작성자 메타데이터 붙이기
- **Qdrant** 적재 : 쪼갠 문단들을 임베딩으로 변환하여 **Qdrant Collection**에 일괄 저장하기
- 필터 검색기 구현: "인사팀 규정 중에서 '연차 사용 방법'과 가장 의미가 통하는 문단 **3**개 찾기" 함수 작성하기
- 답변 연결: 검색해 온 문단을 조합하여 최종 질문-답변 프롬프트 완성해보기